Multibody System Dynamics (2022) 55:137–163
https://doi.org/10.1007/s11044-022-09822-3

Analysis of the model-based corrector approach for explicit cosimulation

Timo Haid[1] · **Daniel Watzenig**[2] · **Georg Stettinger**[3]

Received: 15 September 2021 / Accepted: 18 March 2022 / Published online: 11 April 2022
© The Author(s) 2022

Abstract
In many industries, the focus of testing is currently shifting away from classical hardware tests to the virtual verification and validation of products. To this end, cosimulation has become a common tool for the simulation and analysis of complex systems that span multiple engineering domains and usually involve multiple, heterogeneous and application-specific simulation environments. In particular, the so-called explicit cosimulation allows a widespread application since it has minimal requirements regarding the capabilities of the tool interfaces. However, explicit cosimulation also poses a numerical challenge, especially when the system includes stiff coupling loops. The model-based corrector approach presented in Haid et al. (The 5th Joint International Conference on Multibody System Dynamics, 2018) provides a method for the efficient cosimulation of such systems. In this article, this model-based corrector approach is extended to additional extrapolation methods. By modeling the cosimulation process through a linear recurrence equation and applying it to the two-mass oscillator test model, the influence of model-based correction on the underlying extrapolation methods in terms of stability, accuracy, and error convergence is analyzed. It is shown that adding model-based correction can significantly improve the overall cosimulation, allowing > 10 times larger macrostep sizes or reducing the cosimulation error by a factor of 10 or more in some cases.

Keywords Explicit cosimulation · Model-based correction · Propagation matrix · Stability analysis · Accuracy analysis · Error convergence

✉ T. Haid
timo.haid@porsche.de

D. Watzenig
daniel.watzenig@tugraz.at

G. Stettinger
georg.stettinger@v2c2.at

[1] Department for virtual function development, Dr. Ing. h.c. F. Porsche AG, Porschestraße 911, 71287 Weissach, Germany

[2] Institute of Automation and Control, Graz University of Technology, Inffeldgasse 21/B/I, 8010 Graz, Austria

[3] areaE, Virtual Vehicle Research GmbH, Inffeldgasse 21/A, 8010 Graz, Austria

1 Introduction

Nowadays, many industries are facing increasing product complexity, where a large number of hardware and software components must work together across different engineering domains. While on the one hand, this increases the necessary development effort, on the other hand, development costs have to be reduced, for example by reducing the number of real prototypes and shifting the focus of testing away from classical hardware tests towards the virtual verification and validation of products. In addition, new products must be brought to market in less and less time. To tackle these challenges, a holistic simulation approach that covers all necessary aspects of the final product is necessary. Since in recent years the use of simulation technology has become a common development tool within the individual engineering domains or departments, there are already established, application-specific simulation environments that are usually tailored towards the needs of the individual domain or department. So-called solver coupling or cosimulation offers the possibility to integrate the subsystems of each domain, which are modeled in their respective application-specific simulation environments, into an overall system simulation. This allows a holistic analysis of the overall product behavior, while still exploiting the advantages of the application-specific simulation environments and avoids remodeling of the entire system in a more general, multiphysical simulation tool.

This article starts by giving a short introduction to cosimulation, as well as a rough classification of the general cosimulation approaches and an overview of common methods for explicit cosimulation in Sect. 1. The model-based corrector approach for explicit cosimulation is presented in Sect. 2 and derived for 0th-, 1st-, and 2nd-order polynomial extrapolation. Section 3 introduces the test model and the analysis methodology used in this article. An indepth analysis of the model-based corrector approach when used with 0th-, 1st-, and 2nd-order polynomial extrapolation with regard to stability, accuracy, and error convergence is conducted in Sect. 4. Section 5 takes a brief look at the application of the algorithm to a nonlinear example. Finally, Sect. 6 gives conclusions and an outlook on future research regarding the model-based corrector approach.

1.1 A short introduction to cosimulation

In the solver coupling or cosimulation approach, each subsystem is solved independently for a small time interval (the so-called macrostep size ΔT [7]) by the solver of the respective simulation environment using the subsystem-specific solver step size (the so-called microstep size δT [7]). Therefore, cosimulation is a multirate, multimethod approach. To simulate the behavior of the overall system, the individual subsystems are connected via inputs $\mathbf{u}$ and outputs $\mathbf{y}$, where data is exchanged at and only at the end of each macrostep. As the outputs of a subsystem are generally unknown before the subsystem completes its macrostep, but are usually necessary inputs for the calculation of other subsystems, at least some if not all of these inputs have to be estimated via a suitable extrapolation method (depending on the execution scheme, see Sect. 1.2). As these estimated inputs usually differ from the actual inputs an error is introduced during each macrostep, leading to inaccurate simulation results. Effectively minimizing this error is the basic goal of all the different cosimulation methods available today.

Figure 1 shows a block diagram of a simple cosimulation scenario, where 3 nonlinear subsystems represented in state-space form are interconnected via inputs $\mathbf{u}$ and outputs $\mathbf{y}$. It is easy to see that, for example, in order to calculate subsystem II (denoted by superscript Roman numerals) the outputs $\mathbf{y}^I$ of subsystem I ($\begin{bmatrix} y_1^I & y_2^I & y_2^I \end{bmatrix}^T$) must be known, but since

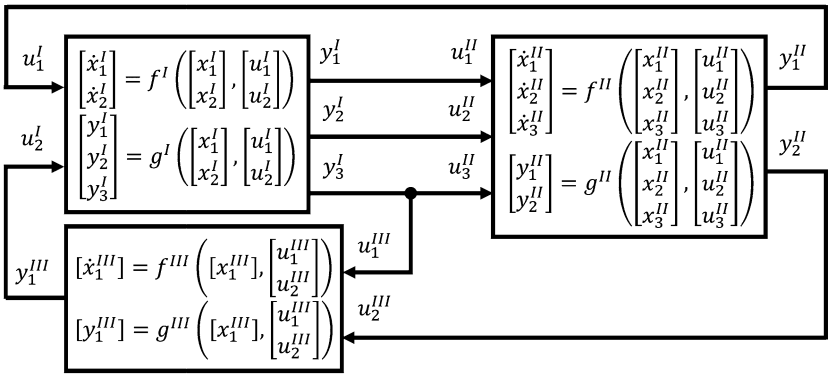


Fig. 1 Block diagram of a simple cosimulation scenario with 3 nonlinear subsystems represented in state-space form

subsystem I in turn also needs outputs from the other two subsystems (y_1^{II} and y_1^{III}) this is not possible without estimating at least some of the corresponding inputs (e.g., u_1^I and u_2^I).

Coordination and interfacing of the subsystems (scheduling, synchronization, and data exchange) is facilitated by a so-called master algorithm. This master algorithm may either be directly included in one of the simulation environments used or provided by a so-called middleware, which is dedicated software that acts as a neutral layer between the simulation environments. Using a dedicated middleware offers advantages in terms of compatibility, usability, and robustness, especially as systems grow more complex (more subsystems and/or a larger number of inputs and outputs per subsystem).

The coupled system shown in Fig. 1 can also be written as a set of differential algebraic equations (DAEs)

$$\begin{bmatrix}\dot{\mathbf{x}}^I\\ \dot{\mathbf{x}}^{II}\\ \dot{\mathbf{x}}^{III}\end{bmatrix} = \begin{bmatrix}f^I\left(\mathbf{x}^I, \mathbf{u}^I\right)\\ f^{II}\left(\mathbf{x}^{II}, \mathbf{u}^{II}\right)\\ f^{III}\left(\mathbf{x}^{III}, \mathbf{u}^{III}\right)\end{bmatrix} \quad \begin{bmatrix}\mathbf{y}^I\\ \mathbf{y}^{II}\\ \mathbf{y}^{III}\end{bmatrix} = \begin{bmatrix}g^I\left(\mathbf{x}^I, \mathbf{u}^I\right)\\ g^{II}\left(\mathbf{x}^{II}, \mathbf{u}^{II}\right)\\ g^{III}\left(\mathbf{x}^{III}, \mathbf{u}^{III}\right)\end{bmatrix} \quad \begin{bmatrix}\mathbf{u}^I\\ \mathbf{u}^{II}\\ \mathbf{u}^{III}\end{bmatrix} = L\begin{bmatrix}\mathbf{y}^I\\ \mathbf{y}^{II}\\ \mathbf{y}^{III}\end{bmatrix}$$
$$\dot{\mathbf{x}} = f\left(\mathbf{x}, \mathbf{u}\right) \qquad \mathbf{y} = g\left(\mathbf{x}, \mathbf{u}\right) \qquad \mathbf{u} = \mathbf{L}\mathbf{y} \quad , \tag{1}$$

where for example $\mathbf{y}^{II}$ denotes the output vector $\begin{bmatrix}y_1^{II} & y_2^{II}\end{bmatrix}^T$, $\mathbf{u}^{II}$ denotes the input vector $\begin{bmatrix}u_1^{II} & u_2^{II} & u_3^{II}\end{bmatrix}^T$, and $\mathbf{x}^{II}$ denotes the state vector $\begin{bmatrix}x_1^{II} & x_2^{II} & x_3^{II}\end{bmatrix}^T$ of subsystem II. The connections between the subsystems are represented by the algebraic equations formed by the coupling matrix $\mathbf{L}$, which is a special selection matrix with exactly one nonzero entry per row (usually 1), since each input of a subsystem is connected to exactly one output (obviously an output can in turn be connected to multiple inputs). For the coupled system shown in Fig. 1 this coupling equation therefore results in:

$$\begin{bmatrix}u_1^I\\ u_2^I\\ u_1^{II}\\ u_2^{II}\\ u_3^{II}\\ u_1^{III}\\ u_2^{III}\end{bmatrix} = \begin{bmatrix}0&0&0&1&0&0\\ 0&0&0&0&0&1\\ 1&0&0&0&0&0\\ 0&1&0&0&0&0\\ 0&0&1&0&0&0\\ 0&0&1&0&0&0\\ 0&0&0&0&1&0\end{bmatrix}\begin{bmatrix}y_1^I\\ y_2^I\\ y_3^I\\ y_1^{II}\\ y_2^{II}\\ y_1^{III}\end{bmatrix}. \tag{2}$$

Note that eqn. (1) represents the coupled system of Fig. 1 without any of the errors introduced by cosimulation, therefore the solution of this DAE would give the true results (the monolithic reference solution without the error introduced by cosimulation). However, since in practical applications the subsystems are simulation models solved by the simulation environments solver this is usually not possible, hence the cosimulation approach. As stated before, the inputs of a subsystem are usually outputs of the other subsystems and therefore unknown before the macrostep is completed and vice versa. In cosimulation, this causality problem is solved by replacing the subsystem inputs $\mathbf{u}$ for the duration of the macrostep with estimated inputs $\tilde{\mathbf{u}}$ calculated by a suitable extrapolation function e that only depends on quantities known before the macrostep, for example, previous outputs or output derivatives. This decouples the DAE eqn. (1) into an ODE for the duration of the macrostep, for example for the nth macrostep $T_n \rightarrow T_{n+1}$:

$$\dot{\mathbf{x}} = f\left(\mathbf{x}, \tilde{\mathbf{u}}\right) \qquad \mathbf{y} = g\left(\mathbf{x}, \tilde{\mathbf{u}}\right) \qquad \tilde{\mathbf{u}} = \mathbf{L}e\left(\mathbf{y}_n, \mathbf{y}_{n-1}, \ldots, \dot{\mathbf{y}}_n, \dot{\mathbf{y}}_{n-1}, \ldots\right) \qquad t \in \left[T_n, T_{n+1}\right]. \tag{3}$$

By applying the different cosimulation methods to eqn. (3) their influence on the overall system behavior can be analyzed.

When the subsystems are linear (or linearized), the DAE of the coupled system (see eqn. (1)) can be written as

$$\dot{\mathbf{x}} = \mathbf{A}\mathbf{x} + \mathbf{B}\mathbf{u} \qquad \mathbf{y} = \mathbf{C}\mathbf{x} + \mathbf{D}\mathbf{u} \qquad \mathbf{u} = \mathbf{L}\mathbf{y}, \tag{4}$$

where the system matrices $\mathbf{A}$, $\mathbf{B}$, $\mathbf{C}$ and $\mathbf{D}$ are block diagonal matrices that consist of the respective subsystem matrices:

$$\mathbf{A} = \begin{bmatrix} \mathbf{A}^I & \ldots & 0 \\ \vdots & \ddots & \vdots \\ 0 & \ldots & \mathbf{A}^N \end{bmatrix} \quad \mathbf{B} = \begin{bmatrix} \mathbf{B}^I & \ldots & 0 \\ \vdots & \ddots & \vdots \\ 0 & \ldots & \mathbf{B}^N \end{bmatrix} \quad \mathbf{C} = \begin{bmatrix} \mathbf{C}^I & \ldots & 0 \\ \vdots & \ddots & \vdots \\ 0 & \ldots & \mathbf{C}^N \end{bmatrix}$$
$$\mathbf{D} = \begin{bmatrix} \mathbf{D}^I & \ldots & 0 \\ \vdots & \ddots & \vdots \\ 0 & \ldots & \mathbf{D}^N \end{bmatrix}. \tag{5}$$

1.2 Classification of cosimulation methods

The different cosimulation approaches available today can be roughly classified by a few key features. These key features are whether a cosimulation method is implicit or explicit, sequential or parallel, and fixed step or variable step.

When each macrostep is calculated only once this leads to a noniterative coupling scheme called explicit cosimulation [20] or weak or inconsistent approach [23]. On the other hand, iteratively repeating macrosteps to improve the solution is commonly referred to as implicit cosimulation [20], Waveform-Iteration [4, 22] or a strong or a consistent approach [21]. Since in explicit cosimulation each macrostep is calculated only once, the error induced due to the extrapolation of the inputs is carried over into the next macrostep. In contrast, the repetition of macrosteps in implicit cosimulation reduces this error with every iteration. As a result, it is quite easy to achieve high stability and accuracy with an implicit cosimulation approach, but it also requires advanced interface capabilities like the reversion of macrosteps. Explicit cosimulation, on the other hand, has minimal requirements regarding

interface capabilities and is therefore applicable for most simulation environments, but,depending on the coupling problem, advanced coupling methods might be necessary to achieve good stability and accuracy.

Depending on whether all subsystem solvers are executed at the same time or one after another, cosimulation methods can also be divided into the parallel Jacobi type and the sequential Gauss–Seidel type [2]. While in a parallel Jacobi-type scheme all inputs have to be extrapolated, in a sequential Gauss–Seidel-type scheme some inputs are already known, since the respective subsystems have already calculated the macrostep and the respective outputs are known. As a result, sequential cosimulation is usually more accurate since it requires less extrapolation, which is the major source of error but also requires more time to calculate since only one subsystem is performing its macrostep at any given time.

Finally, the macrostep size used can be either fixed throughout the simulation or dynamically adjusted by a macrostep-size controller (see, e.g., [5, 6, 16, 29, 30]). Using a fixed macrostep size is a simple and robust approach, but can also lead to unnecessarily small macrostep sizes in phases with low system dynamics. By using a variable macrostep size larger macrostep sizes can be used during such phases, which usually improves the simulation performance. On the downside, there is a risk that the step-size controller cannot adjust the macrostep size in time if the system dynamics suddenly change, causing large errors or instability in the simulation.

For the purpose of this article, an explicit parallel cosimulation with a constant macrostep size is assumed in order to reduce the complexity of the analysis. Nevertheless, it is easily possible to apply the coupling approaches and methodologies presented in this article to sequential and/or variable step coupling schemes.

1.3 Cosimulation methods for explicit cosimulation

Probably the most popular cosimulation approach for explicit cosimulation is to extrapolate the inputs via a polynomial of sufficient order p, where a range from zero to two is most common [10, 11]. While higher-order polynomials can achieve higher accuracy and better error convergence, they are also more susceptible to oscillations, also known as Runge's phenomenon, which leads to stability problems for relatively large macrostep sizes with respect to the signal frequency and decreases the region of both numerical and zero stability [11]. In addition, the subsystem interfaces involved must be able to process these higher-order polynomials by adjusting the input values for each microstep (and each intermediate step in the case of a multistep method) of the subsystem solver according to this extrapolation polynomial. For polynomial extrapolation either macrosteps, microsteps, exact derivatives, or a combination of these can be used. While access to macrosteps is a basic capability of all subsystem interfaces used for cosimulation, access to microsteps or even exact derivatives of the outputs is rarer, but if available, the use of microsteps or exact derivatives can often increase the accuracy of the extrapolation [7].

While (higher-order) polynomial extrapolation adjusts the input values only as a function of time, more advanced approaches like Semi-Implicit Coupling (SIC) [32] or Linear Implicit Stabilization (LIS) [3] also adjust the inputs as a function of the subsystem outputs. To this end, SIC uses a first-order multivariable Taylor polynomial with the corresponding directional derivatives to obtain a first-order approximation of the subsystem inputs with respect to the outputs. On the other hand, LIS uses a local linear representation of the residual system in ODE form, which is solved in conjunction with the subsystem itself by the subsystem solver. Both, SIC and LIS, require model knowledge in the form of directional derivatives, and in the case of LIS, access to the subsystem states is also necessary. Since

both methods must be embedded in the subsystems, the implementation of the subsystem interfaces can be challenging.

Another method that uses model knowledge is the so-called Model-Based Coupling approach (MBC) [33] originally introduced for real-time cosimulation with test benches or hardware-in-the-loop systems. Since real-time applications involve additional delay due to signal latency or signal loss, MBC uses a local linear model to predict the subsystem inputs of multiple future macrosteps. The linear model is identified during runtime using either Kalman Filters or Recursive Least Squares [34].

The coupling error caused by the difference between the extrapolated inputs and the actual inputs can also be interpreted as residual energy, which is introduced during each macrostep. Since after each macrostep the actual inputs are known, this residual energy can be estimated. Energy-conservation methods like the Nearly Energy Preserving Coupling Element (NEPCE) [7] compensate this residual energy by adding an appropriate correction signal to the extrapolated inputs during the next macrosteps. An extension to NEPCE that also takes the direct feedthrough of the subsystems into account is given in [28]. Naturally, these methods can correct the residual energy only after the fact. The Model-Based Pre-Step Stabilization (MBStab) [13] on the other hand uses model knowledge to approximate the monolithic reference solution and then optimizes the extrapolated inputs such that the difference between the approximated reference solution and the cosimulation becomes minimal. Therefore, this approach already compensates the residual energy during the macrostep in which it is produced, which leads to very good performance in terms of stability and accuracy.

Since the master algorithm is responsible for exchanging the subsystem outputs after each macrostep, these outputs can be adjusted during the data exchange to correct for the error in the extrapolated inputs. Parallel Linear Correction (PLC) [1] estimates and corrects for the error that was transferred directly to the outputs due to direct feedthrough. The Model-Based Corrector (MBCorr) [17] takes an extended but similar approach and is explained in detail in the next section.

2 The model-based corrector approach for explicit cosimulation

As stated before, the extrapolation of the inputs required for explicit cosimulation induces an error to the overall system with every macrostep. The model-based corrector approach for explicit cosimulation addresses this problem by estimating this induced error and correcting the outputs of the subsystems accordingly after every macrostep. Since the states of the subsystems usually cannot be accessed by the master algorithm in practical applications, the error introduced to the states is corrected during the next macrostep by applying a constant correction signal δu_c to the subsystem inputs. This two-step process is shown in Fig. 2.

2.1 Correcting the error in the outputs

For the correction of the outputs $\mathbf{y}_c = \mathbf{y} + \boldsymbol{\Delta}\mathbf{y}$, again consider the nonlinear coupled system eqn. (1). For this section, all true inputs, outputs, and states of the coupled system are denoted by an overline. The exactly solved system that is therefore unaffected by any cosimulation approach can be written as:

$$\dot{\overline{\mathbf{x}}} = f(\overline{\mathbf{x}}, \overline{\mathbf{u}}) \qquad \overline{\mathbf{y}} = g(\overline{\mathbf{x}}, \overline{\mathbf{u}}) \qquad \overline{\mathbf{u}} = \mathbf{L}\overline{\mathbf{y}}. \tag{6}$$

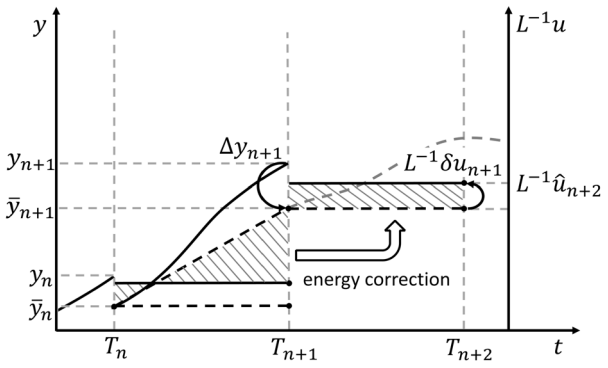


Fig. 2 The two-step process of the model-based corrector approach

This nonlinear coupled system is then linearized for the duration of the current macrostep $T_n \rightarrow T_{n+1}$:

$$\begin{aligned}
\dot{\overline{\mathbf{x}}} &= \left.\frac{\partial f}{\partial x}\right|_n \overline{\mathbf{x}} + \left.\frac{\partial f}{\partial u}\right|_{\mathbf{n}} \overline{\mathbf{u}} \quad &= \mathbf{A}_n\overline{\mathbf{x}} + \mathbf{B}_n\overline{\mathbf{u}} \\
\overline{\mathbf{y}} &= \left.\frac{\partial g}{\partial x}\right|_n \overline{\mathbf{x}} + \left.\frac{\partial g}{\partial u}\right|_n \overline{\mathbf{u}} \quad &= \mathbf{C}_n\overline{\mathbf{x}} + \mathbf{D}_n\overline{\mathbf{u}} \\
\overline{\mathbf{u}} &= \mathbf{L}\overline{\mathbf{y}}.
\end{aligned} \tag{7}$$

Note that in practice it is necessary that the subsystems supply their Jacobians $\left.\frac{\partial f^N}{\partial x^N}\right|_n$, $\left.\frac{\partial f^N}{\partial u^N}\right|_n$, $\left.\frac{\partial g^N}{\partial x^N}\right|_n$ and $\left.\frac{\partial g^N}{\partial u^N}\right|_n$ to the master algorithm after every macrostep. Alternatively, system identification methods like Multivariable Output Error State Space identification (MOESP) [19, 25] or Recursive Least Squares (RLS) [24] can be used in order to acquire the subsystem Jacobians. The master algorithm then combines the subsystem Jacobians into the block diagonal matrices $\mathbf{A}_n$, $\mathbf{B}_n$, $\mathbf{C}_n$ and $\mathbf{D}_n$, respectively, (see eqn (5)).

With the extrapolated inputs $\tilde{\mathbf{u}} = \mathbf{L}e(\mathbf{y}_n, \ldots, \mathbf{y}_{n-p})$ the corresponding cosimulated system can be written in linearized form as well by using the same Jacobians:

$$\begin{aligned}
\dot{\mathbf{x}} &= \left.\frac{\partial f}{\partial x}\right|_n \mathbf{x} + \left.\frac{\partial f}{\partial u}\right|_n \tilde{\mathbf{u}} \quad &= \mathbf{A}_n\mathbf{x} + \mathbf{B}_n\tilde{\mathbf{u}} \\
\mathbf{y} &= \left.\frac{\partial g}{\partial x}\right|_n \mathbf{x} + \left.\frac{\partial g}{\partial u}\right|_n \tilde{\mathbf{u}} \quad &= \mathbf{C}_n\mathbf{x} + \mathbf{D}_n\tilde{\mathbf{u}} \\
\tilde{\mathbf{u}} &= \mathbf{L}e(\mathbf{y}_n, \ldots, \mathbf{y}_{n-p}).
\end{aligned} \tag{8}$$

Subtracting eqn. (8) from eqn. (7) gives the error in the states $\boldsymbol{\Delta}\mathbf{x} = \overline{\mathbf{x}} - \mathbf{x}$ and the outputs $\boldsymbol{\Delta}\mathbf{y} = \overline{\mathbf{y}} - \mathbf{y}$ due to the cosimulation approach:

$$\begin{aligned}
\dot{\overline{\mathbf{x}}} - \dot{\mathbf{x}} &= \mathbf{A}_n\left(\overline{\mathbf{x}} - \mathbf{x}\right) + \mathbf{B}_n\left(\overline{\mathbf{u}} - \tilde{\mathbf{u}}\right) \\
\overline{\mathbf{y}} - \mathbf{y} &= \mathbf{C}_n\left(\overline{\mathbf{x}} - \mathbf{x}\right) + \mathbf{D}_n\left(\overline{\mathbf{u}} - \tilde{\mathbf{u}}\right).
\end{aligned} \tag{9}$$

The error in the outputs at the end of the macrostep $T_n \rightarrow T_{n+1}$ can directly be calculated as:

$$\boldsymbol{\Delta}\mathbf{y}_{n+1} = \overline{\mathbf{y}}_{n+1} - \mathbf{y}_{n+1} = \mathbf{C}_n\left(\overline{\mathbf{x}}_{n+1} - \mathbf{x}_{n+1}\right) + \mathbf{D}_n\left(\overline{\mathbf{u}}_{n+1} - \tilde{\mathbf{u}}_{n+1}\right). \tag{10}$$

The error in the states at the end of the macrostep ($\overline{\mathbf{x}}_{n+1} - \mathbf{x}_{n+1}$) can be calculated by solving the state error differential equation eqn. (9) for the duration of the macrostep $T_n \rightarrow T_{n+1}$. The solution of this type of ODE is given, for example, in [26], for eqn. (9) this results in:

$$\overline{\mathbf{x}}_{n+1} - \mathbf{x}_{n+1} = e^{\mathbf{A}_n \Delta T} \left(\overline{\mathbf{x}}_n - \mathbf{x}_n\right) + \int_{T_n}^{T_{n+1}} e^{\mathbf{A}_n (T_{n+1} - \tau)} \mathbf{B}_n \left(\overline{\mathbf{u}}(\tau) - \tilde{\mathbf{u}}(\tau)\right) d\tau. \tag{11}$$

Since the model-based corrector approach only corrects for the error induced during the current macrostep (the local error), the state error at the beginning of the macrostep $\overline{\mathbf{x}}_n - \mathbf{x}_n$ is assumed to be 0, which eliminates the first term of eqn. (11). To solve the convolution integral in the second term, both the true inputs $\overline{\mathbf{u}}$ and the extrapolated inputs $\tilde{\mathbf{u}}$ are approximated by a Taylor-series expansion of sufficient order p. For example, for the extrapolated inputs this Taylor series is given as:

$$\begin{aligned} \tilde{\mathbf{u}}(\tau) &= \tilde{\mathbf{u}}_n + \frac{\dot{\tilde{\mathbf{u}}}_n}{1}(\tau - T_n) + \frac{\ddot{\tilde{\mathbf{u}}}_n}{2}(\tau - T_n)^2 + \frac{\dddot{\tilde{\mathbf{u}}}_n}{6}(\tau - T_n)^3 + \ldots + \frac{\tilde{\mathbf{u}}_n^{(p)}}{p!}(\tau - T_n)^p \\ &= \sum_{m=0}^{p} \frac{\tilde{\mathbf{u}}^{(m)}}{m!}(\tau - T_n)^m. \end{aligned} \tag{12}$$

As a result, eqn. (11) can be written as:

$$\begin{aligned} \overline{\mathbf{x}}_{n+1} - \mathbf{x}_{n+1} = \quad & \mathbf{A}_n^{-1} \left(e^{\mathbf{A}_n \Delta T} - \mathbf{I}\right) \mathbf{B}_n \left(\overline{\mathbf{u}}_n - \tilde{\mathbf{u}}_n\right) \\ + \quad & \mathbf{A}_n^{-2} \left(e^{\mathbf{A}_n \Delta T} - \mathbf{I} - \mathbf{A}_n \Delta T\right) \mathbf{B}_n \left(\dot{\overline{\mathbf{u}}}_n - \dot{\tilde{\mathbf{u}}}_n\right) \\ + \quad & \mathbf{A}_n^{-3} \left(e^{\mathbf{A}_n \Delta T} - \mathbf{I} - \mathbf{A}_n \Delta T - \frac{1}{2}\mathbf{A}_n^2 \Delta T^2\right) \mathbf{B}_n \left(\ddot{\overline{\mathbf{u}}}_n - \ddot{\tilde{\mathbf{u}}}_n\right) \\ & \vdots \\ + \quad & \mathbf{A}_n^{-(p+1)} \left(e^{\mathbf{A}_n \Delta T} - \mathbf{I} - \mathbf{A}_n \Delta T - \ldots - \frac{1}{p!}\mathbf{A}_n^p \Delta T^p\right) \mathbf{B}_n \left(\overline{\mathbf{u}}_n^{(p)} - \tilde{\mathbf{u}}_n^{(p)}\right) \\ = \quad & \mathbf{B}_{d_0,n} \left(\overline{\mathbf{u}}_n - \tilde{\mathbf{u}}_n\right) + \mathbf{B}_{d_1,n} \left(\dot{\overline{\mathbf{u}}}_n - \dot{\tilde{\mathbf{u}}}_n\right) + \ldots + \mathbf{B}_{d_p,n} \left(\overline{\mathbf{u}}_n^{(p)} - \tilde{\mathbf{u}}_n^{(p)}\right). \end{aligned} \tag{13}$$

By inserting eqn. (13) back into eqn. (10) and using the Taylor-series expansions for $\overline{\mathbf{u}}$ and $\tilde{\mathbf{u}}$ at T_{n+1}, the estimate for the local error in the outputs can be calculated as

$$\begin{aligned} \overline{\mathbf{y}}_{n+1} - \mathbf{y}_{n+1} = & \left(\mathbf{C}_n \mathbf{B}_{d_0,n} + \mathbf{D}_n\right) \left(\overline{\mathbf{u}}_n - \tilde{\mathbf{u}}_n\right) + \left(\mathbf{C}_n \mathbf{B}_{d_1,n} + \mathbf{D}_n \Delta T\right) \left(\dot{\overline{\mathbf{u}}}_n - \dot{\tilde{\mathbf{u}}}_n\right) \\ & + \ldots + \left(\mathbf{C}_n \mathbf{B}_{d_p,n} + \mathbf{D}_n \frac{\Delta T^p}{p!}\right) \left(\overline{\mathbf{u}}_n^{(p)} - \tilde{\mathbf{u}}_n^{(p)}\right), \end{aligned} \tag{14}$$

where the extrapolated inputs and their derivatives depend on the actual cosimulation method used, while the true inputs and their derivatives must to be approximated via a suitable method.

For now, let us assume that the extrapolated inputs are constant for the duration of the macrostep and equal to the corresponding true outputs of the previous macrostep plus the constant correction offset ($\tilde{\mathbf{u}}_n = \mathbf{L}\overline{\mathbf{y}}_n + \boldsymbol{\delta}\mathbf{u}_{c,n}$ and $\dot{\tilde{\mathbf{u}}}_n \ldots \tilde{\mathbf{u}}_n^{(p)} = 0$), which results in the simplest cosimulation approach, the so-called Zero-Order Hold (ZOH). By assuming the true

inputs as piecewise constant as well and equal to the corresponding true outputs of the current macrostep ($\overline{\mathbf{u}}_n = \mathbf{L}\overline{\mathbf{y}}_{n+1}$ and $\dot{\overline{\mathbf{u}}}_n \ldots \overline{\mathbf{u}}_n^{(p)} = 0$), eqn. (14) can be written as

$$\overline{\mathbf{y}}_{n+1} - \mathbf{y}_{n+1} = \left(\mathbf{C}_n\mathbf{B}_{d_0,n} + \mathbf{D}_n\right)\left(\mathbf{L}\overline{\mathbf{y}}_{n+1} - \left(\mathbf{L}\overline{\mathbf{y}}_n + \boldsymbol{\delta}\mathbf{u}_{c,n}\right)\right), \tag{15}$$

which can be solved for the true outputs as

$$\begin{aligned}\overline{\mathbf{y}}_{n+1} &= \left(\mathbf{I} - \left(\mathbf{C}_n\mathbf{B}_{d_0,n} + \mathbf{D}_n\right)\mathbf{L}\right)^{-1}\left(\mathbf{y}_{n+1} - \left(\mathbf{C}_n\mathbf{B}_{d_0,n} + \mathbf{D}_n\right)\left(\mathbf{L}\overline{\mathbf{y}}_n + \boldsymbol{\delta}\mathbf{u}_{c,n}\right)\right)\\ &= \left(\mathbf{I} - \mathbf{G}_{ZOH,n}\mathbf{L}\right)^{-1}\left(\mathbf{y}_{n+1} - \mathbf{G}_{ZOH,n}\mathbf{L}\overline{\mathbf{y}}_n - \mathbf{G}_{ZOH,n}\boldsymbol{\delta}\mathbf{u}_{c,n}\right),\end{aligned} \tag{16}$$

where $\mathbf{G}_{ZOH,n}$ denotes the correction matrix for ZOH for the nth macrostep. By looking at the definitions of the output error $\boldsymbol{\Delta}\mathbf{y} = \overline{\mathbf{y}} - \mathbf{y}$ and the output correction $\mathbf{y}_c = \mathbf{y} + \boldsymbol{\Delta}\mathbf{y}$ it can be seen that the true outputs calculated by eqn. (16) are actually the desired corrected outputs for the macrostep.

If instead of ZOH the extrapolation function is a 1st-order polynomial this leads to a so-called First-Order Hold (FOH) approach. By constructing this 1st-order extrapolation polynomial through the outputs of the two previous macrosteps $\overline{\mathbf{y}}_n$ and $\overline{\mathbf{y}}_{n-1}$ (which is the most common approach) and by approximating the true outputs also with a 1st-order polynomial through $\overline{\mathbf{y}}_{n+1}$ and $\overline{\mathbf{y}}_n$, the extrapolated and true inputs plus their derivatives result in:

$$\begin{aligned}\tilde{\mathbf{u}}_n &= \mathbf{L}\overline{\mathbf{y}}_n + \boldsymbol{\delta}\mathbf{u}_{c,n} \qquad & \dot{\tilde{\mathbf{u}}}_n &= \mathbf{L}\left(\frac{\overline{\mathbf{y}}_n - \overline{\mathbf{y}}_{n-1}}{\Delta T}\right) \qquad & \ddot{\tilde{\mathbf{u}}}_n \ldots \tilde{\mathbf{u}}_n^{(p)} &= 0\\ \overline{\mathbf{u}}_n &= \mathbf{L}\overline{\mathbf{y}}_n & \dot{\overline{\mathbf{u}}}_n &= \mathbf{L}\left(\frac{\overline{\mathbf{y}}_{n+1} - \overline{\mathbf{y}}_n}{\Delta T}\right) & \ddot{\overline{\mathbf{u}}}_n \ldots \overline{\mathbf{u}}_n^{(p)} &= 0.\end{aligned} \tag{17}$$

Inserting both into eqn. (14) and again solving for the true outputs (aka the corrected outputs) at the end of the current macrostep $\overline{\mathbf{y}}_{n+1}$ yields

$$\begin{aligned}\overline{\mathbf{y}}_{n+1} = &\left(\mathbf{I} - \left(\frac{\mathbf{C}_n\mathbf{B}_{d_1,n}}{\Delta T} + \mathbf{D}_n\right)\mathbf{L}\right)^{-1}\left(\mathbf{y}_{n+1} - \left(\frac{\mathbf{C}_n\mathbf{B}_{d_1,n}}{\Delta T} + \mathbf{D}_n\right)\mathbf{L}\left(2\overline{\mathbf{y}}_n - \overline{\mathbf{y}}_{n-1}\right)\right.\\ &\left. - \left(\mathbf{C}_n\mathbf{B}_{d_0,n} + \mathbf{D}_n\right)\boldsymbol{\delta}\mathbf{u}_{c,n}\right)\\ = &\left(\mathbf{I} - \mathbf{G}_{FOH,n}\mathbf{L}\right)^{-1}\left(\mathbf{y}_{n+1} - \mathbf{G}_{FOH,n}\mathbf{L}\left(2\overline{\mathbf{y}}_n - \overline{\mathbf{y}}_{n-1}\right) - \mathbf{G}_{ZOH,n}\boldsymbol{\delta}\mathbf{u}_{c,n}\right),\end{aligned} \tag{18}$$

where $\mathbf{G}_{FOH,n}$ denotes the correction matrix for FOH for the nth macrostep. For a higher-order polynomial extrapolation approach the same process can be repeated in order to obtain the appropriate calculation rule for the corrected outputs. For example, for Second-Order Hold (SOH) this calculation rule can be given as:

$$\begin{aligned}\overline{\mathbf{y}}_{n+1} &= \left(\mathbf{I} - \mathbf{G}_{SOH,n}\mathbf{L}\right)^{-1}\left(\mathbf{y}_{n+1} - \mathbf{G}_{SOH,n}\mathbf{L}\left(3\overline{\mathbf{y}}_n - 3\overline{\mathbf{y}}_{n-1} + \overline{\mathbf{y}}_{n-2}\right) - \mathbf{G}_{ZOH,n}\boldsymbol{\delta}\mathbf{u}_{c,n}\right)\\ \mathbf{G}_{SOH,n} &= \frac{\mathbf{C}_n\mathbf{B}_{d_1,n}}{2\Delta T} + \frac{\mathbf{C}_n\mathbf{B}_{d_2,n}}{\Delta T^2} + \mathbf{D}_n.\end{aligned} \tag{19}$$

Note that the model-based corrector approach is only applicable if eqn. (14) can be solved for the corrected outputs $\overline{\mathbf{y}}_{n+1}$ for the actual cosimulation method, which might not be possible in all cases. Furthermore, the calculation rule for the corrected outputs does not need to depend exclusively on the previous outputs, it can also depend on any known quantity.

2.2 Correcting the error in the states

As can be seen from eqn. (11), the difference between the true inputs $\overline{\mathbf{u}}$ and the extrapolated inputs $\tilde{\mathbf{u}}$ accumulates an error in the states during the macrostep. As stated before, the accumulated error in the states at the end of the macrostep cannot be corrected directly, since in practical applications the states of the subsystems are usually not directly accessible to the master algorithm. Instead, the error in the states is corrected similar to an energy-correction scheme, by applying a constant correction signal $\boldsymbol{\delta}\mathbf{u}_c$ to the extrapolated inputs during the next macrostep (see [7] and [28]). To this end, the accumulated error $\mathbf{E}_r$ induced by the extrapolated inputs is tracked over the course of the simulation, which can be calculated for the current macrostep $T_n \rightarrow T_{n+1}$ as:

$$\mathbf{E}_{r_{n+1}} = \mathbf{E}_{r_n} + \int_{T_n}^{T_{n+1}} \left(\overline{\mathbf{u}}(\tau) - \tilde{\mathbf{u}}(\tau)\right) d\tau. \tag{20}$$

A certain fraction α of this accumulated error is then compensated during the next macrostep by choosing a correction signal $\boldsymbol{\delta}\mathbf{u}$ such that

$$\int_{T_n+1}^{T_{n+2}} \boldsymbol{\delta}\mathbf{u}(\tau) d\tau = \alpha \mathbf{E}_{r_{n+1}}. \tag{21}$$

Since the model-based corrector approach uses a constant correction offset $\boldsymbol{\delta}\mathbf{u}_c$ for each macrostep (therefore $\Delta T \boldsymbol{\delta}\mathbf{u}_{c,n+1} = \alpha \mathbf{E}_{r_{n+1}}$, $\Delta T \boldsymbol{\delta}\mathbf{u}_{c,n} = \alpha \mathbf{E}_{r_n}$, etc.), inserting eqn. (20) into eqn. (21) yields the constant correction offset for the next macrostep $\boldsymbol{\delta}\mathbf{u}_{c,n+1}$:

$$\begin{aligned} \boldsymbol{\delta}\mathbf{u}_{c,n+1} &= \frac{\alpha}{\Delta T}\mathbf{E}_{r_n} + \frac{\alpha}{\Delta T}\int_{T_n}^{T_{n+1}} \left(\overline{\mathbf{u}}(\tau) - \tilde{\mathbf{u}}(\tau)\right) d\tau \\ &= \boldsymbol{\delta}\mathbf{u}_{c,n} + \frac{\alpha}{\Delta T}\int_{T_n}^{T_{n+1}} \left(\overline{\mathbf{u}}(\tau) - \tilde{\mathbf{u}}(\tau)\right) d\tau. \end{aligned} \tag{22}$$

Note that setting $\alpha = 1$ is the most common choice that results in instant correction of the accumulated error during the next macrostep, but it might also be helpful to over- or undercompensate in some cases.

The true and extrapolated inputs in eqn. (22) are again approximated via a Taylor-series expansion (see eqn. (12)) yielding

$$\boldsymbol{\delta}\mathbf{u}_{c,n+1} = \boldsymbol{\delta}\mathbf{u}_{c,n} + \alpha \left(\left(\overline{\mathbf{u}}_n - \tilde{\mathbf{u}}_n\right) + \frac{\Delta T}{2}\left(\dot{\overline{\mathbf{u}}}_n - \dot{\tilde{\mathbf{u}}}_n\right) + \ldots + \frac{\Delta T^{(p)}}{(p+1)!}\left(\overline{\mathbf{u}}_n^{(p)} - \tilde{\mathbf{u}}_n^{(p)}\right) \right), \tag{23}$$

where again the extrapolated inputs and their derivatives depend on the actual cosimulation method used, while the true inputs and their derivatives must be approximated via a suitable method.

For example, for ZOH the extrapolated inputs and their derivatives are again given as $\tilde{\mathbf{u}}_n = \mathbf{L}\overline{\mathbf{y}}_n + \boldsymbol{\delta}\mathbf{u}_{c,n}$ and $\dot{\tilde{\mathbf{u}}}_n \ldots \tilde{\mathbf{u}}_n^{(p)} = 0$, while in contrast to the output correction the true inputs are approximated with a 1st-order polynomial though $\overline{\mathbf{y}}_{n+1}$ and $\overline{\mathbf{y}}_n$ (see eqn. (17)), which results in:

$$\boldsymbol{\delta}u_{c,n+1} = \boldsymbol{\delta}\mathbf{u}_{c,n} + \alpha \left(\mathbf{L}\left(\frac{\overline{\mathbf{y}}_{n+1} - \overline{\mathbf{y}}_n}{2}\right) - \boldsymbol{\delta}\mathbf{u}_{c,n} \right). \tag{24}$$

Similarly, for FOH the extrapolated inputs are given by a 1st-order polynomial, while the true inputs are approximated by a 2nd-order polynomial, which results in:

$$\boldsymbol{\delta}\mathbf{u}_{c,n+1} = \boldsymbol{\delta}\mathbf{u}_{c,n} + \alpha \left(\mathbf{L} \left(\frac{5\overline{\mathbf{y}}_{n+1} - 10\overline{\mathbf{y}}_n + 5\overline{\mathbf{y}}_{n-1}}{12} \right) - \boldsymbol{\delta}\mathbf{u}_{c,n} \right). \tag{25}$$

Again, deriving the correction offset for other extrapolation methods follows the same process.

2.3 Summary of the model-based corrector algorithm

Finally, the model-based corrector approach consists of the following steps, which are performed by the master algorithm after every macrostep:

1. Collect the current outputs of each subsystem in the output vector $\mathbf{y}_{n+1}$.
2. Collect the Jacobians of each subsystem either through a specialized interface (e.g., FMI 2.0 [8]) or via system identification (e.g., MOESP [19, 25]) and compile $\mathbf{A}_n$, $\mathbf{B}_n$, $\mathbf{C}_n$ and $\mathbf{D}_n$ (see eqn. (5)).
3. Calculate the corrected outputs $\overline{\mathbf{y}}_{n+1}$ via the corresponding solution of eqn. (14), for example in the case of ZOH extrapolation eqn. (16).
4. Calculate the correction offset $\boldsymbol{\delta}\mathbf{u}_{c,n+1}$ via the corresponding solution of eqn. (23), for example in the case of ZOH extrapolation eqn. (24).
5. Propagate the corrected outputs $\overline{\mathbf{y}}_{n+1}$ and the correction offset $\boldsymbol{\delta}\mathbf{u}_{c,n+1}$ to the next macrostep, for example in the case of ZOH calculate the extrapolated inputs for the next macrostep as $\tilde{\mathbf{u}}_{n+2} = \mathbf{L}\overline{\mathbf{y}}_{n+1} + \boldsymbol{\delta}\mathbf{u}_{c,n+1}$.

Since the model-based corrector approach only requires access to the inputs and outputs of the subsystems, at least when used in conjunction with a system-identification method, it is applicable to a wide variety of practical applications, as this is a basic capability that virtually all cosimulation interfaces provide. If a subsystem provides advanced capabilities like access to the subsystem Jacobians, these may be used as well. Either way, exotic interface capabilities such as direct access to the subsystem states are not required.

3 Test model and analysis methodology

To analyze the stability, accuracy, and error convergence of the model-based corrector approach and compare it to other coupling methods, the well-known two-mass oscillator with force–displacement coupling is used as a test model. This test model can be interpreted as an extension of Dahlquists test equation [18] commonly used for the analysis of numerical integration methods to cosimulation [11]. Finally, such a linear test model allows for highly efficient analysis, since in this case the cosimulation process for most cosimulation methods can be described by a linear recurrence relation of the form

$$\begin{bmatrix} \mathbf{x}_{n+1}^T & \mathbf{y}_{n+1}^T & \mathbf{y}_n^T & \dots \end{bmatrix}^T = \boldsymbol{\Phi} \begin{bmatrix} \mathbf{x}_n^T & \mathbf{y}_n^T & \mathbf{y}_{n-1}^T & \dots \end{bmatrix}^T$$
$$\mathbf{z}_{n+1} = \boldsymbol{\Phi}\mathbf{z}_n, \tag{26}$$

where the so-called propagation matrix $\boldsymbol{\Phi}$ advances the solution of the cosimulated system from one macrostep to the next, resulting in a discrete formulation of the cosimulated system's behavior, where the discrete steps represent the macrosteps of the cosimulation. Note

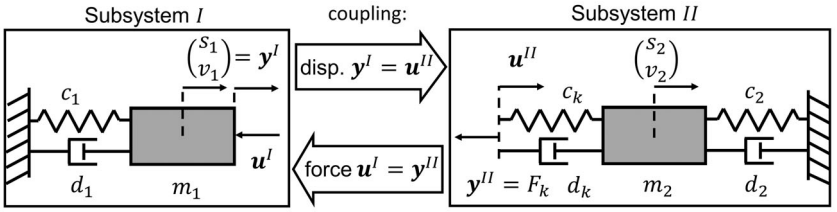


Fig. 3 Two-mass oscillator test model separated by a force–displacement coupling approach (see, e.g., [12])

that the subsystem ODEs are solved exactly and therefore the cosimulation approach is the only source of error. As expected, the propagation matrix changes depending on the cosimulation method, the subsystem parameters, and the macrostep size ΔT. All necessary data is collected in the information vector $\mathbf{z}$, which represents the overall state of the cosimulated system after each macrostep. Using this propagation-matrix approach makes the analysis of the cosimulation stability, accuracy, and error convergence highly efficient since the cosimulation process is fully described by a series of matrix multiplications in which the subsystems are always solved exactly without the need to choose a sufficiently small microstep size as in classical numerical cosimulation.

3.1 The two-mass oscillator test model

The linear two-mass oscillator with force–displacement coupling shown in Fig. 3 is probably the most commonly used test model in cosimulation related contributions in the literature (see [1, 7, 9, 12, 14, 15, 31] among others). As shown in Fig. 3, the two-mass oscillator is split into two subsystems at the connection of mass m_1 and the coupling spring damper element c_k, d_k. Therefore, subsystem I provides the displacement $s_1 = y_1^I$ and velocity $v_1 = y_2^I$ of mass m_1 to subsystem II, which returns the coupling force $F_k = y_1^{II}$ (see Sect. 1.1 for details on the notation). The DAE for the coupled two-mass oscillator then results in (see eqn. (4)):

$$
\begin{aligned}
\begin{bmatrix} \dot{x}_1^I \\ \dot{x}_2^I \\ \dot{x}_1^{II} \\ \dot{x}_2^{II} \end{bmatrix} &= \begin{bmatrix} 0 & 1 & & \\ -\frac{c_1}{m_1} & -\frac{d_1}{m_1} & & \\ & & 0 & 1 \\ & & -\frac{c_k+c_2}{m_2} & -\frac{d_k+d_2}{m_2} \end{bmatrix} \begin{bmatrix} x_1^I \\ x_2^I \\ x_1^{II} \\ x_2^{II} \end{bmatrix} + \begin{bmatrix} 0 & & \\ \frac{1}{m_1} & & \\ & 0 & 0 \\ & \frac{c_k}{m_2} & \frac{d_k}{m_2} \end{bmatrix} \begin{bmatrix} u_1^I \\ u_1^{II} \\ u_2^{II} \end{bmatrix} \\
\begin{bmatrix} y_1^I \\ y_2^I \\ y_1^{II} \end{bmatrix} &= \begin{bmatrix} 1 & 0 & & \\ 0 & 1 & & \\ & & c_k & d_k \end{bmatrix} \begin{bmatrix} x_1^I \\ x_2^I \\ x_1^{II} \\ x_2^{II} \end{bmatrix} + \begin{bmatrix} 0 & & \\ 0 & & \\ & -c_k & -d_k \end{bmatrix} \begin{bmatrix} u_1^I \\ u_1^{II} \\ u_2^{II} \end{bmatrix} \\
\begin{bmatrix} u_1^I \\ u_1^{II} \\ u_2^{II} \end{bmatrix} &= \begin{bmatrix} 0 & 0 & 1 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} y_1^I \\ y_2^I \\ y_1^{II} \end{bmatrix}.
\end{aligned}
\tag{27}
$$

For the purpose of this article the parameters of the two single-mass oscillators are set to $m_1 = m_2 = 10kg$, $c_1 = 1e^6 \frac{N}{m}$, $c_2 = 1e^7 \frac{N}{m}$, $d_1 = 1 \frac{Ns}{m}$ and $d_2 = 2 \frac{Ns}{m}$, while the parameters of the coupling spring damper element c_k, d_k and the macrostep size ΔT are varied in the range $c_k \in \left[1, 1e^{10}\right], d_k \in \left[1, 1e^7\right]$ and $\Delta T \in \left[3e^{-3}, 1e^{-6}\right]$.

3.2 Cosimulation stability in the time domain

As mentioned before, cosimulation induces an error that can lead to the instability of the system. Therefore, a fundamental requirement for the applicability of a cosimulation method for a given problem is the stability of the cosimulated system.

Given a set of initial conditions (an initial information vector $\mathbf{z}_0$), the linear recurrence relation eqn. (26) completely defines the solution of the cosimulated system, for example for n macrosteps (from T_0 to T_n) as $\mathbf{z}_n = \boldsymbol{\Phi}^n \mathbf{z}_0$. If the resulting information vector is bound by some real value ε for an infinite number of macrosteps, the cosimulated system is stable and, since the coupled system itself is assumed to be stable, therefore the cosimulation approach is stable, which leads to the following condition:

$$\lim_{n\to\infty} \|\boldsymbol{\Phi}^n \mathbf{z}_0\| \leq \varepsilon. \tag{28}$$

In order for the power sequence of the propagation matrix $\boldsymbol{\Phi}$ to be bound, the spectral radius of $\boldsymbol{\Phi}$ has to be either less than or equal to 1, meaning all eigenvalues of $\boldsymbol{\Phi}$ have to be inside the unit circle:

$$\rho(\boldsymbol{\Phi}) = max(|eig(\boldsymbol{\Phi})|) \leq 1. \tag{29}$$

Note that multiple eigenvalues exactly on the unit circle ($\lambda_{1..n} = 1$) must be nondefective, meaning their algebraic and geometric multiplicity must be the same. By evaluating eqn. (29) it can be directly seen whether a cosimulation method is stable for the given coupled system and macrostep size. A basic requirement for any cosimulation method to be feasible is to fulfill eqn. (29) at least as the macrostep size approaches 0. This property is called zero stability [21].

3.3 Cosimulation accuracy in the time domain

While stability is a basic prerequisite for any cosimulation to be feasible, stability alone does not automatically imply good results. For example, consider a cosimulation where all subsystems themselves are stable. Then, a cosimulation method that just sets all coupling signals to 0 would provide a stable cosimulation for arbitrarily large macrostep sizes. Therefore, the accuracy of the cosimulated system must also be considered. A straightforward approach is to calculate a reference solution and compare it to the results of the cosimulation. In the case of a linear test model such as the two-mass oscillator this reference solution can also be written as a linear recurrence equation similar to eqn. (26) as

$$\mathbf{z}^*_{n+1} = \boldsymbol{\Phi}^* \mathbf{z}^*_n, \tag{30}$$

where $\boldsymbol{\Phi}^*$ denotes the propagation matrix and $\mathbf{z}^*$ denotes the information vector of the reference solution (denoted by a superscript $*$). By assuming a common information vector $\mathbf{z}_n$ at the beginning of the macrostep, the local error of the current macrostep $\boldsymbol{\varepsilon}_{l,n+1}$ can be directly calculated:

$$\boldsymbol{\varepsilon}_{l,n+1} := \boldsymbol{\Phi}^* \mathbf{z}_n - \boldsymbol{\Phi} \mathbf{z}_n = \left(\boldsymbol{\Phi}^* - \boldsymbol{\Phi}\right) \mathbf{z}_n. \tag{31}$$

By using separate information vectors for the cosimulated and reference solutions the global error for the current macrostep $\boldsymbol{\varepsilon}_{g,n+1}$ can be calculated as:

$$\boldsymbol{\varepsilon}_{g,n+1} := \boldsymbol{\Phi}^* \mathbf{z}^*_n - \boldsymbol{\Phi} \mathbf{z}_n = \boldsymbol{\Phi}^* \boldsymbol{\Phi}^{*^n} \mathbf{z}_0 - \boldsymbol{\Phi} \boldsymbol{\Phi}^n \mathbf{z}_0 = \left(\boldsymbol{\Phi}^{*^{n+1}} - \boldsymbol{\Phi}^{n+1}\right) \mathbf{z}_0. \tag{32}$$

Equations (31) and (32) produce an error vector with an error value for every element of the information vector (all states, outputs, previous outputs, etc.) for every macrostep. The resulting $k \times n$ error matrix produced by a cosimulation with n macrosteps has to be reduced by an appropriate error norm. In this article, a normalized root mean square error (NRMSE) is used to combine the macrosteps into a single normalized error vector, where the empirical standard deviation is used for normalization. The resulting normalized error vector is then combined into a scalar error indicator via a root mean square:

$$e_{NRMSE} = \sqrt{\frac{1}{k}\sum_{j=1}^{k} \varepsilon_{j,NRMSE}^2} \quad \text{with} \quad \varepsilon_{j,NRMSE} = \frac{\sqrt{\frac{1}{n}\sum_{i=1}^{n} \varepsilon_{j,i}^2}}{\sqrt{\frac{1}{n}\sum_{i=1}^{n} (z_{j,i}^* - \frac{1}{n}\sum_{i=1}^{n} z_{j,i}^*)^2}}. \tag{33}$$

Since in practical applications the states of the subsystems are not available, only the outputs of the current macrostep are included in the calculation of the error indicator in this article.

To obtain the propagation matrix for the reference solution $\boldsymbol{\Phi}^*$, the coupling equation is inserted into both the state differential and the output equation in the linear coupled system DAE eqn. (4), resulting in:

$$\begin{aligned} \dot{\mathbf{x}} &= \mathbf{A}\mathbf{x} + \mathbf{B}\mathbf{L}\mathbf{y} \\ \mathbf{y} &= (\mathbf{I} - \mathbf{D}\mathbf{L})^{-1}\mathbf{C}\mathbf{x}. \end{aligned} \tag{34}$$

Inserting this modified output equation in the state differential equation yields the coupled system DAE in the form of an autonomous linear system ODE, since the coupling equation is now implicitly included:

$$\begin{aligned} \dot{\mathbf{x}} &= (\mathbf{A} + \mathbf{B}\mathbf{L}(\mathbf{I} - \mathbf{D}\mathbf{L})^{-1}\mathbf{C})\mathbf{x} &&= \mathbf{A}^*\mathbf{x} \\ \mathbf{y} &= (\mathbf{I} - \mathbf{D}\mathbf{L})^{-1}\mathbf{C}\mathbf{x} &&= \mathbf{C}^*\mathbf{x}. \end{aligned} \tag{35}$$

The solution of this ODE for the duration of a macrostep $T_n \rightarrow T_{n+1}$ is once again given in [26] (see Sect. 2.1) as

$$\begin{aligned} \mathbf{x}_{n+1} &= e^{\mathbf{A}^*\Delta T}\mathbf{x}_n &&= \mathbf{A}_d^*\mathbf{x}_n \\ \mathbf{y}_{n+1} &= (\mathbf{I} - \mathbf{D}\mathbf{L})^{-1}\mathbf{C}\mathbf{x}_{n+1} &&= \mathbf{C}^*\mathbf{A}_d^*\mathbf{x}_n, \end{aligned} \tag{36}$$

which can be written in propagation matrix form as:

$$\begin{aligned} \begin{bmatrix} \mathbf{x}_{n+1} \\ \mathbf{y}_{n+1} \end{bmatrix} &= \begin{bmatrix} \mathbf{A}_d^* & 0 \\ \mathbf{C}^*\mathbf{A}_d^* & 0 \end{bmatrix} \begin{bmatrix} \mathbf{x}_n \\ \mathbf{y}_n \end{bmatrix} \\ \mathbf{z}_{n+1}^* &= \boldsymbol{\Phi}^*\mathbf{z}_n. \end{aligned} \tag{37}$$

Note that in the reference solution the next macrostep depends solely on the previous states $\mathbf{x}_n$ and therefore the columns corresponding to the previous outputs $\mathbf{y}_n$ are zero but are kept to match the dimension of the information vector shared with the cosimulation approach. As expected, since $\rho(\boldsymbol{\Phi}^*) = \rho(\mathbf{A}_d^*)$, the exact solution is stable for all macrostep sizes if the underlying system is stable.

4 Analysis of the model-based corrector approach

As shown in Sect. 2, the model-based corrector approach is always used in conjunction with a specific extrapolation method. In this section, the influence of model-based correction on

the stability, accuracy, and error convergence of polynomial extrapolation (ZOH, FOH, and SOH) is analyzed. To apply the methodology presented in Sect. 3 for the analysis of the respective polynomial extrapolation approaches with and without model-based correction, the corresponding propagation matrices have to be determined.

Again, starting with the linear coupled system DAE eqn. (4), replacing the inputs $\mathbf{u}$ with extrapolated inputs $\tilde{\mathbf{u}}$ and solving the resulting state ODE for the duration of a macrostep $T_n \rightarrow T_{n+1}$ in the same way as the state error differential equation eqn. (9) (see Sect. 2.1) yields:

$$\begin{aligned}
\mathbf{x}_{n+1} &= \mathbf{A}_d \mathbf{x}_n + \mathbf{B}_{d_0} \tilde{\mathbf{u}}_n + \mathbf{B}_{d_1} \dot{\tilde{\mathbf{u}}}_n + \ldots + \mathbf{B}_{d_p} \tilde{\mathbf{u}}_n^{(p)} \\
\mathbf{y}_{n+1} &= \mathbf{C}\mathbf{A}_d \mathbf{x}_n + \left(\mathbf{C}\mathbf{B}_{d_0} + \mathbf{D}\right) \tilde{\mathbf{u}}_n + \left(\mathbf{C}\mathbf{B}_{d_1} + \mathbf{D}\Delta T\right) \dot{\tilde{\mathbf{u}}}_n + \ldots + \left(\mathbf{C}\mathbf{B}_{d_p} + \mathbf{D}\frac{\Delta T^p}{p!}\right) \tilde{\mathbf{u}}_n^{(p)}.
\end{aligned} \tag{38}$$

Again, the extrapolated inputs $\tilde{\mathbf{u}}_n$ and their derivatives $\dot{\tilde{\mathbf{u}}}_n \ldots \tilde{\mathbf{u}}_n^{(p)}$ depend on the actual cosimulation method used.

For example, for the model-based corrector approach when used with ZOH, the extrapolated inputs for ZOH $\tilde{\mathbf{u}}_n = \mathbf{L}\overline{\mathbf{y}}_n + \boldsymbol{\delta}\mathbf{u}_{c,n}$ and their derivatives $\dot{\tilde{\mathbf{u}}}_n \ldots \tilde{\mathbf{u}}_n^{(p)} = 0$ are inserted into eqn. (38):

$$\begin{aligned}
\mathbf{x}_{n+1} &= \mathbf{A}_d \mathbf{x}_n + \mathbf{B}_{d_0} \mathbf{L}\overline{\mathbf{y}}_n + \mathbf{B}_{d_0} \boldsymbol{\delta}\mathbf{u}_{c,n} \\
\mathbf{y}_{n+1} &= \mathbf{C}\mathbf{A}_d \mathbf{x}_n + \left(\mathbf{C}\mathbf{B}_{d_0} + \mathbf{D}\right) \mathbf{L}\overline{\mathbf{y}}_n + \left(\mathbf{C}\mathbf{B}_{d_0} + \mathbf{D}\right) \boldsymbol{\delta}\mathbf{u}_{c,n} \\
&= \mathbf{C}\mathbf{A}_d \mathbf{x}_n + \mathbf{K}_y \mathbf{L}\overline{\mathbf{y}}_n + \mathbf{K}_y \boldsymbol{\delta}\mathbf{u}_{c,n}.
\end{aligned} \tag{39}$$

Inserting the resulting output equation into the output correction equation for ZOH (eqn. (16)) yields

$$\begin{aligned}
\overline{\mathbf{y}}_{n+1} &= \left(\mathbf{I} - \mathbf{G}_{ZOH}\mathbf{L}\right)^{-1} \left(\mathbf{C}\mathbf{A}_d \mathbf{x}_n + \left(\mathbf{K}_y \mathbf{L} - \mathbf{G}_{ZOH}\mathbf{L}\right) \overline{\mathbf{y}}_n + \left(\mathbf{K}_y - \mathbf{G}_{ZOH}\right) \boldsymbol{\delta}\mathbf{u}_{c,n}\right) \\
&= \mathbf{M}_x \mathbf{x}_n + \mathbf{M}_y \overline{\mathbf{y}}_n + \mathbf{M}_{\delta u} \boldsymbol{\delta}\mathbf{u}_n,
\end{aligned} \tag{40}$$

which can then be inserted into the calculation rule for the correction offset $\boldsymbol{\delta}\mathbf{u}_{c,n+1}$ for ZOH (eqn. (24)) resulting in:

$$\begin{aligned}
\boldsymbol{\delta}\mathbf{u}_{c,n+1} &= \frac{\alpha}{2}\mathbf{L}\mathbf{M}_x \mathbf{x}_n + \frac{\alpha}{2}\mathbf{L}\left(\mathbf{M}_y - \mathbf{I}\right)\overline{\mathbf{y}}_n + \left(\mathbf{I} + \frac{\alpha}{2}\left(\mathbf{L}\mathbf{M}_{\delta u} - 2\mathbf{I}\right)\right)\boldsymbol{\delta}\mathbf{u}_{c,n} \\
&= \mathbf{H}_x \mathbf{x}_n + \mathbf{H}_y \overline{\mathbf{y}}_n + \mathbf{H}_{\delta u} \boldsymbol{\delta}\mathbf{u}_{c,n}.
\end{aligned} \tag{41}$$

The equations above can be combined in the propagation matrix for the model-based corrector approach used with ZOH as:

$$\begin{bmatrix} \mathbf{x}_{n+1} \\ \overline{\mathbf{y}}_{n+1} \\ \boldsymbol{\delta}\mathbf{u}_{c,n+1} \end{bmatrix} = \begin{bmatrix} \mathbf{A}_d & \mathbf{B}_{d_0}\mathbf{L} & \mathbf{B}_{d_0} \\ \mathbf{M}_x & \mathbf{M}_y & \mathbf{M}_{\delta u} \\ \mathbf{H}_x & \mathbf{H}_y & \mathbf{H}_{\delta u} \end{bmatrix} \begin{bmatrix} \mathbf{x}_n \\ \overline{\mathbf{y}}_n \\ \boldsymbol{\delta}\mathbf{u}_{c,n} \end{bmatrix} \tag{42}$$

$$\mathbf{z}_{n+1} = \boldsymbol{\Phi}_{ZOH_MBCorr} \cdot \mathbf{z}_n.$$

By setting the correction matrix $\mathbf{G}_{ZOH}$ to 0 and eliminating the correction offset $\boldsymbol{\delta}\mathbf{u}$ by setting $\mathbf{H}_x$, $\mathbf{H}_y$ and $\mathbf{H}_{\delta u}$ to 0, the propagation matrix for ZOH without model-based correction can directly be seen from eqn. (42).

Note that in the case of linear subsystems with perfect model knowledge $\mathbf{K}_y = \mathbf{G}_{ZOH}$ and therefore $\mathbf{M}_y = \mathbf{M}_{\delta u} = 0$, which effectively eliminates the influence of the previous outputs $\overline{\mathbf{y}}_n$ and the correction offset $\boldsymbol{\delta}\mathbf{u}_{c,n}$ in the calculation of the current outputs $\overline{\mathbf{y}}_{n+1}$. Therefore, any error in the previous outputs is not directly propagated into the next macrostep. The distinction between $\mathbf{K}_y$ and $\mathbf{G}_{ZOH}$ is kept due to the difference in meaning and to include imperfect model knowledge. The propagation matrices for FOH or SOH with and without model-based correction are derived in the same way by using the respective formulations for the extrapolated inputs, their derivatives plus the output correction and the correction offset.

4.1 Stability of the model-based corrector approach

As described in Sect. 3, the coupling stiffness c_k and damping d_k of the two-mass oscillator (see Sect. 3.1) are varied in ranges $c_k \in \left[1, 10^{10}\right]$, $d_k \in \left[1, 10^{7}\right]$ for several macrostep sizes ΔT in the range $\Delta T \in \left[10^{-2.5}, 10^{-6}\right]$. The stability limit where $\rho(\boldsymbol{\Phi}) = 1$ (see Sect. 3.2) is marked by a corresponding contour line for each macrostep size ΔT. The macrostep sizes ΔT are uniformly distributed in logarithmic space such that there is a factor of $\sqrt{10}$ between adjacent macrostep sizes, with a corresponding gray scale starting from black (largest) to light gray (smallest). To cover the wide range of values all three axis (c_k, d_k and ΔT) are in logarithmic scale.

Figure 4 shows the resulting contour plots that allow an easy analysis of the influence of model-based correction on the stability regions of the different extrapolation methods. In addition, Table 1 lists the macrostep size ΔT, where $\rho(\boldsymbol{\Phi}) = 1$ for several combinations of c_k and d_k.

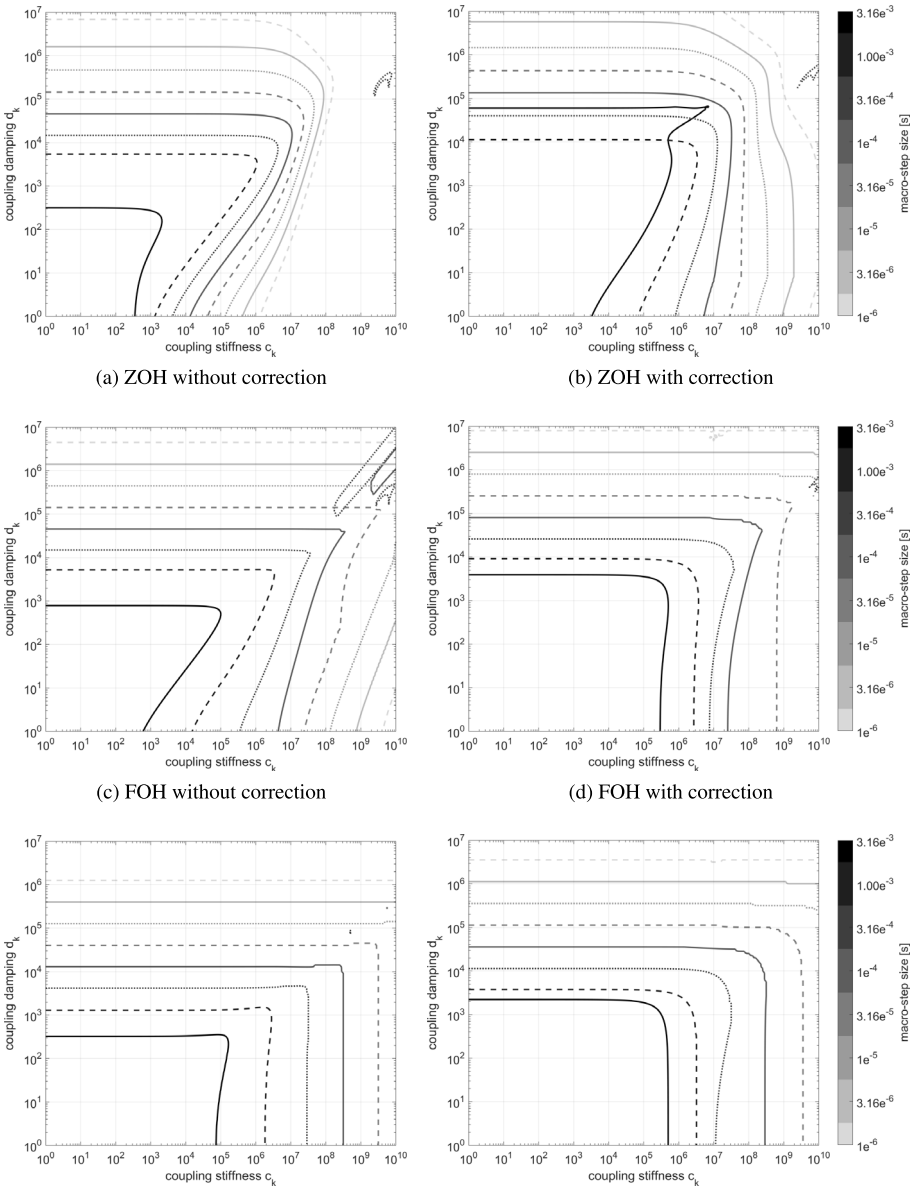
As can be seen in the lower-right quarter of Fig. 4(b), adding model-based correction to ZOH largely improves stability for combinations of high coupling stiffness c_k and low coupling damping d_k. Furthermore, there is a large improvement in stability for relatively large macrostep sizes along both the c_k and d_k axis (see lower-left quarter). Finally, there is also some improvement in stability for large damping, low stiffness combinations.

Figure 4(d) shows the effect of model-based correction on FOH extrapolation, which is in principle very similar to ZOH (large improvement for high c_k, low d_k or large ΔT; some improvement for high d_k), but overall the improvement is not as large as for ZOH.

Finally, it can be seen from Fig. 4(f) that the stability improvement for high coupling stiffness, low coupling damping combinations due to model-based correction has almost disappeared for SOH, but there is still some improvement for high coupling damping scenarios.

As a result, adding model-based correction to polynomial extrapolation can greatly improve stability for high coupling stiffness, low coupling damping combinations, or when relatively large macrostep sizes are used, especially for low polynomial orders. Furthermore, model-based correction also improves stability for high coupling damping and higher polynomial orders.

It can also be seen in Fig. 4 that in some cases there are small stable areas for high stiffness, high damping scenarios for relatively large macrostep sizes (most prominent in Fig. 4(c)). Furthermore, in the case of ZOH with correction (Fig. 4(b)) the stability for high damping is better for the largest macrostep size used than for the two next smaller macrostep sizes. Since the local error introduced by the cosimulation does not necessarily work towards the destabilization of the system, but might also have a stabilizing effect in some cases, the spectral radius for a given combination of c_k and d_k does not monotonically decrease as the macrostep size decreases. For example, the spectral radius $\rho(\Phi)$ with respect to the macrostep size ΔT for FOH without correction (see Fig. 4(b)) and $c_k = 10^9$, $d_k =$



(a) ZOH without correction

(b) ZOH with correction

(c) FOH without correction

(d) FOH with correction

(e) SOH without correction

(f) SOH with correction

Fig. 4 Stability regions for ZOH, FOH and SOH with and without model-based correction

$3.6 \cdot 10^5$ is shown in Fig. 5. It can be seen that the spectral radius dips below 1 around $\Delta T = 3.16 \cdot 10^{-4}$, which is the same macrostep size that exhibits a small stable area in Fig. 4(b) for this combination of c_k and d_k. The practical consequence of this effect is that the assumption that given a stable macrostep size for a certain case all smaller macrostep sizes are also stable is false in these cases.

Table 1 Largest stable macrostep size for ZOH, FOH, and SOH with and without model-based correction for several combinations of c_k and d_k

Coupling stiffness c_k	$1.0e^{1}$	$1.0e^{1}$	$1.0e^{1}$	$1.0e^{5}$	$1.0e^{5}$	$1.0e^{5}$	$1.0e^{9}$	$1.0e^{9}$	$1.0e^{9}$
Coupling damping d_k	$1.0e^{1}$	$3.2e^{3}$	$1.0e^{6}$	$1.0e^{1}$	$3.2e^{3}$	$1.0e^{6}$	$1.0e^{1}$	$3.2e^{3}$	$1.0e^{6}$
ZOH without correction	$3.5e^{-3}$	$1.8e^{-3}$	$4.9e^{-6}$	$7.2e^{-5}$	$1.7e^{-3}$	$4.8e^{-6}$	N/A	N/A	N/A
ZOH with correction	$1.0e^{-2}$	$6.7e^{-3}$	$1.4e^{-5}$	$1.5e^{-3}$	$7.0e^{-3}$	$1.4e^{-5}$	$4.9e^{-6}$	$3.7e^{-6}$	$6.3e^{-7}$
ZOH macrostep gain	2.9	3.7	2.9	21.1	4.0	2.9	N/A	N/A	N/A
FOH without correction	$4.8e^{-3}$	$1.4e^{-3}$	$4.7e^{-6}$	$9.3e^{-4}$	$1.4e^{-3}$	$4.7e^{-6}$	$4.8e^{-6}$	$2.1e^{-5}$	$4.7e^{-6}$
FOH with correction	$9.5e^{-3}$	$3.1e^{-3}$	$8.1e^{-6}$	$3.8e^{-3}$	$2.9e^{-3}$	$8.1e^{-6}$	$2.7e^{-5}$	$2.7e^{-5}$	$8.0e^{-6}$
FOH macrostep gain	2.0	2.2	1.7	4.1	2.0	1.7	5.7	1.3	1.7
SOH without correction	$5.9e^{-3}$	$4.1e^{-4}$	$1.3e^{-6}$	$3.1e^{-3}$	$4.1e^{-4}$	$1.3e^{-6}$	$5.7e^{-5}$	$5.7e^{-5}$	$1.3e^{-6}$
SOH with correction	$9.4e^{-3}$	$1.2e^{-3}$	$3.6e^{-6}$	$4.4e^{-3}$	$1.1e^{-3}$	$3.6e^{-6}$	$6.2e^{-5}$	$6.0e^{-5}$	$3.5e^{-6}$
SOH macrostep gain	1.6	2.9	2.7	1.4	2.8	2.7	1.1	1.0	2.7

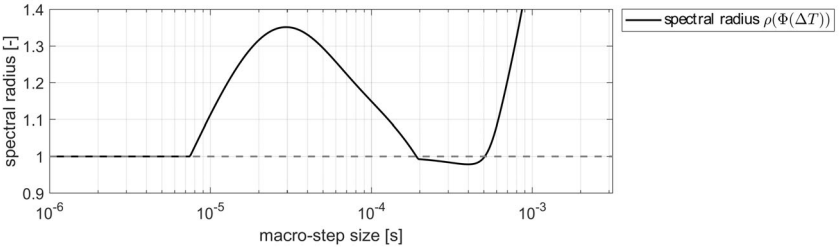


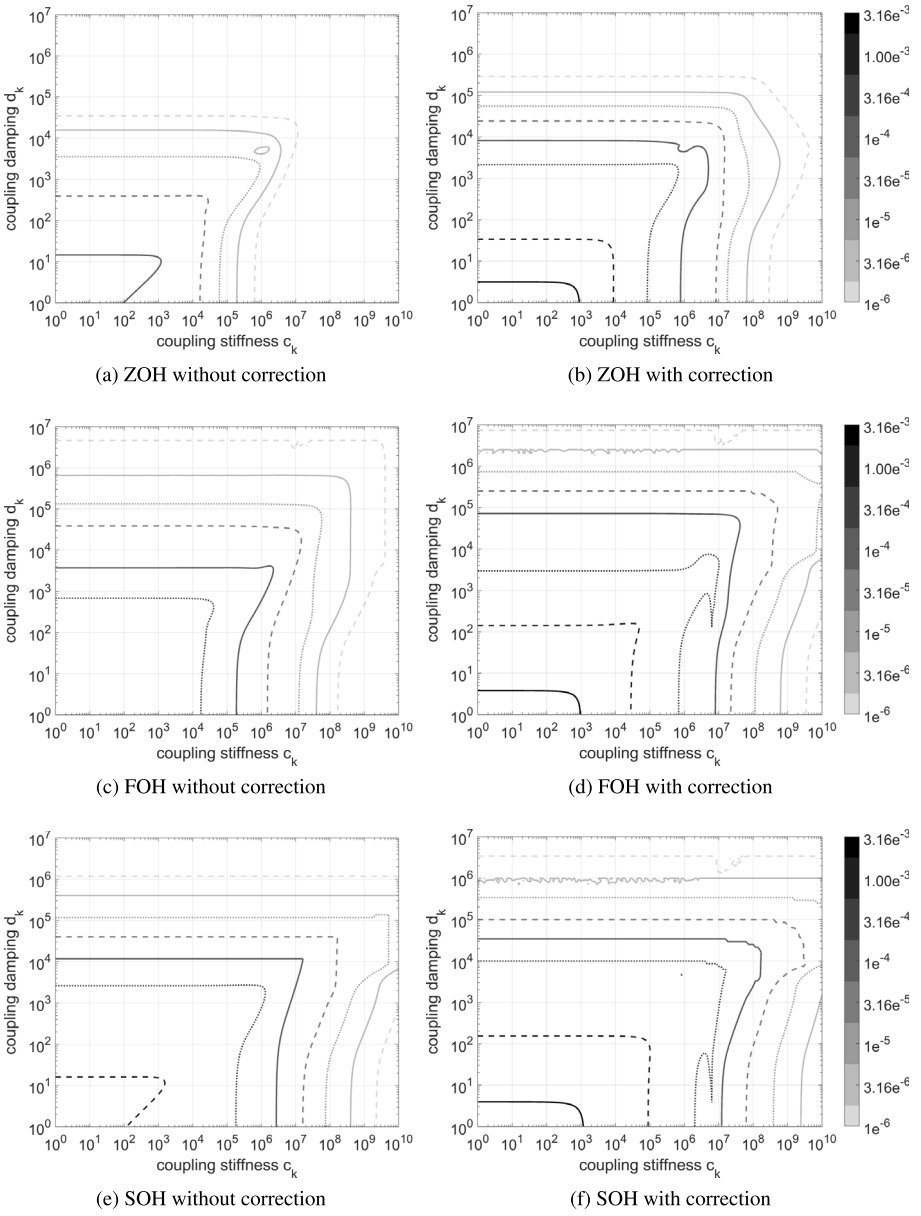
Fig. 5 Spectral radius $\rho(\Phi)$ with respect to the macrostep size ΔT for FOH without correction at $c_k = 10^9$, $d_k = 3.6 \cdot 10^5$

Also note that since the model-based corrector approach corrects the local error induced during each macrostep, the amount of correction and therefore the improvement due to model-based correction depends on the effectiveness of the underlying extrapolation method. The better the underlying extrapolation method performs for the actual case, the less local error it induces during each macrostep, and the less correction has to be applied by the model-based corrector leading to a smaller improvement. For example, it can be seen in Fig. 4 that for low coupling damping d_k the stability improves with higher polynomial order, therefore the improvement due to model-based correction gets smaller as the polynomial order increases in this case.

4.2 Accuracy of the model-based corrector approach

For the analysis of the influence of model-based correction on the accuracy of polynomial extrapolation methods, again the coupling stiffness c_k and damping d_k of the two-mass oscillator (see Sect. 3.1) are varied in the ranges $c_k \in \left[1, 10^{10}\right], d_k \in \left[1, 10^{7}\right]$ for several macrostep sizes ΔT in the range $\Delta T \in \left[10^{-2.5}, 10^{-6}\right]$ in the same way as for the stability analysis (see Sect. 4.1).

Unlike the stability limit, the accuracy limit is not fixed, but a matter of definition. In this analysis, the cosimulation is considered accurate if the global normalized root mean square error indicator $e_{g,NRMSE}$ (see Sect. 3.3) is below a threshold of $e_{g,NRMSE} \leq 0.01$. The error



(a) ZOH without correction

(b) ZOH with correction

(c) FOH without correction

(d) FOH with correction

(e) SOH without correction

(f) SOH with correction

Fig. 6 Accuracy regions for ZOH, FOH and SOH with and without model-based correction

threshold was chosen in such a way that the signal plots of the subsystem outputs of both the cosimulation and the reference solution are indistinguishable without zooming in, which represents a practical choice for the acceptable error in many industrial applications (assuming the limits on the y-axis are the signal min and max values and the x-axis is reasonably scaled, e.g., from 0 s to 0.3 s in this case). The accuracy limit where $e_{g,NRMSE} = 0.01$ is marked by a corresponding contour line for each macrostep size ΔT. Figure 6 shows the

Table 2 Largest accurate macrostep size for ZOH, FOH, and SOH with and without model-based correction for several combinations of c_k and d_k

Coupling stiffness c_k	$1.0e^{1}$	$1.0e^{1}$	$1.0e^{1}$	$3.2e^{4}$	$3.2e^{4}$	$3.2e^{4}$	$1.0e^{8}$	$1.0e^{8}$	$1.0e^{8}$
Coupling damping d_k	$1.0e^{1}$	$1.0e^{3}$	$1.0e^{5}$	$1.0e^{1}$	$1.0e^{3}$	$1.0e^{5}$	$1.0e^{1}$	$1.0e^{3}$	$1.0e^{5}$
ZOH without correction	$1.1e^{-4}$	$2.5e^{-5}$	$1.0e^{-7}$	$1.9e^{-5}$	$2.6e^{-5}$	$1.0e^{-7}$	$1.0e^{-7}$	$1.0e^{-7}$	$1.0e^{-7}$
ZOH with correction	$1.7e^{-3}$	$4.6e^{-4}$	$4.2e^{-6}$	$5.3e^{-4}$	$4.7e^{-4}$	$4.2e^{-6}$	$2.4e^{-6}$	$8.5e^{-6}$	$2.2e^{-6}$
ZOH macrostep gain	16.3	18.2	42.5	27.6	18.3	42.5	23.9	85.5	21.9
FOH without correction	$5.8e^{-4}$	$3.0e^{-4}$	$1.3e^{-5}$	$2.5e^{-4}$	$3.0e^{-4}$	$1.3e^{-5}$	$1.6e^{-6}$	$6.1e^{-6}$	$6.7e^{-6}$
FOH with correction	$2.2e^{-3}$	$5.0e^{-4}$	$7.6e^{-5}$	$1.0e^{-3}$	$5.1e^{-4}$	$7.6e^{-5}$	$1.1e^{-5}$	$3.9e^{-5}$	$5.4e^{-5}$
FOH macrostep gain	3.8	1.7	5.8	4.0	1.7	5.8	7.1	6.4	8.0
SOH without correction	$1.0e^{-3}$	$4.6e^{-4}$	$1.3e^{-5}$	$5.9e^{-4}$	$4.6e^{-4}$	$1.3e^{-5}$	$8.5e^{-6}$	$2.9e^{-5}$	$1.3e^{-5}$
SOH with correction	$2.0e^{-3}$	$6.8e^{-4}$	$3.6e^{-5}$	$1.3e^{-3}$	$6.9e^{-4}$	$3.6e^{-5}$	$2.4e^{-5}$	$6.3e^{-5}$	$3.5e^{-5}$
SOH macrostep gain	2.0	1.5	2.7	2.3	1.5	2.7	2.9	2.2	2.6

resulting contour plots that allow easy analysis of the influence of model-based correction on the accuracy regions of the different extrapolation methods. Again, all three axis (c_k, d_k and ΔT) are in logarithmic scale. In addition, Table 2 lists the macrostep size ΔT where $e_{g,NRMSE} = 0.01$ for several combinations of c_k and d_k.

As can be seen from Fig. 6(b), similar to stability adding model-based correction to ZOH leads to a large accuracy improvement for high coupling stiffness, low coupling damping combinations (lower-right quarter), and some accuracy improvements for high coupling damping (upper-left quarter). Furthermore, model-based correction greatly improves accuracy for large macrostep sizes. It can be seen that without model-based correction there are no accurate solutions for macrostep sizes of $\Delta T \leq 3.16 \cdot 10^{-4}$ s, while there is already a substantial accuracy region for $\Delta T = 3.16 \cdot 10^{-4}$ s when model-based correction is applied. To achieve the same accuracy region without model-based correction a macrostep size of about $\Delta T = 1 \cdot 10^{-5}$ s is necessary.

Figure 6(d) shows the effect of model-based correction on FOH extrapolation. Again, there is a large improvement for high c_k, low d_k combinations or large macrostep sizes and also some improvement for high damping. Similar to stability, the accuracy improvement for FOH is not as large as for ZOH

Finally, it can be seen from Fig. 6(f) that in contrast to stability, there is still some accuracy improvement for high coupling stiffness, low coupling damping combinations when model-based correction is applied to SOH, although the effect is further reduced compared to FOH or ZOH extrapolation. Also, there is an accuracy improvement for large macrostep sizes and high damping scenarios, but again the effect is reduced compared to lower-order polynomial extrapolation in both cases.

It can be seen in Fig. 6 (e.g., (a), (b), (d), and (f)) that for certain macrostep sizes there are bulges, dents, or branches in the contour line. This is caused by the fact that for each coupling method analyzed in this section, the global error reaches a local minimum for a certain coupling stiffness c_k. Changes in the coupling damping d_k or the macrostep size ΔT move this local minimum mostly up and down as they change the magnitude of the global error. Therefore, in most cases, this local minimum is either completely above or below the defined accuracy limit and thus invisible, but when it is close to the defined accuracy limit, it causes the observed effects, depending on the exact shape and position. Figure 7 depicts the global error $e_{g,NRMSE}$ with respect to the coupling stiffness c_k for SOH with correction (see

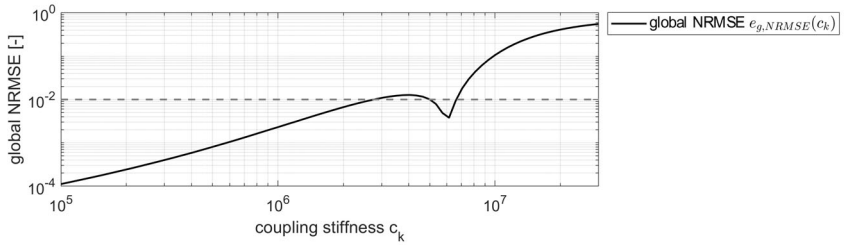


Fig. 7 Global error $e_{g,NRMSE}$ with respect to the coupling stiffness c_k for SOH with correction at $d_k = 40$, $\Delta T = 3.16 \cdot 10^{-4}$

Fig. 6(f)) and $d_k = 40$, $\Delta T = 3.16 \cdot 10^{-4}$. It can be seen that the local minimum of the global error is right next to the accuracy limit of $e_{g,NRMSE} = 0.01$, causing the branch observed for a macrostep size of $\Delta T = 3.16 \cdot 10^{-4}$ in Fig. 6(f). In contrast to the effects observed for stability, given an accurate macrostep size for a certain case all smaller macrostep sizes are also accurate since the error is monotonically decreasing with decreasing macrostep size once below a certain threshold (see Sect. 4.3). Also, note that the jitter observed in Figs. 6(f) and (d) for very high coupling damping and very small macrostep sizes is caused by the fact that the calculation of the propagation matrix becomes ill-conditioned for these cases.

As a result, adding model-based correction to polynomial extrapolation can greatly improve accuracy for high coupling stiffness, low coupling damping combinations, or when relatively large macrostep sizes are used, especially for low polynomial orders. Furthermore, model-based correction also improves accuracy for high coupling damping and higher polynomial orders. This is the same observation as for stability, although in general, the improvement in accuracy due to model-based correction is larger than the improvement in stability. Also, note that the correlation between higher polynomial order and smaller improvement due to model-based correction is much clearer for accuracy than for stability, since the correlation between higher-order polynomial extrapolation and improved accuracy is much clearer in this case.

4.3 Error convergence of the model-based corrector approach

Finally, the influence of model-based correction on the convergence of the coupling error as the macrostep size is reduced shall be analyzed. To this end, the coupling stiffness c_k and the coupling damping d_k are fixed to moderate values of $c_k = 2 \cdot 10^5$ and $d_k = 5 \cdot 10^2$, respectively. The macrostep size ΔT is varied again in the range $\Delta T \in \left[3 \cdot 10^{-3}, 1 \cdot 10^{-6}\right]$ and the resulting local normalized root mean square error indicator $e_{l,NRMSE}$ (see Sect. 3.3) is observed in log–log scale. In addition, Table 3 shows the local error for several given macrostep sizes as well as the necessary macrostep size for several desired local errors for each method.

Figure 8 shows the convergence of the local error for polynomial extrapolation for 0th order to 4th order with and without model-based correction. It can be seen that adding model-based correction increases the error convergence order of the cosimulation such that the convergence of the next higher polynomial is achieved. For example, ZOH with model-based correction achieves the same error-convergence order as FOH without model-based correction, and so on. In addition, applying model-based correction to polynomial extrapolation also achieves a constant offset regarding the local error (in log scale) compared to the next higher polynomial extrapolation order. As a result, adding model-based correction to a

Table 3 Local error for a given macrostep size and necessary macrostep size for a desired local error for 0th-order to 4th-order polynomial extrapolation with and without model-based correction

macrostep size ΔT	$1.0e^{-3}$	$1.0e^{-4}$	$1.0e^{-5}$			
local error $e_{l,NRMSE}$				$1.0e^{-2}$	$1.0e^{-5}$	$1.0e^{-8}$
0th Order Hold	$7.2e^{0}$	$1.6e^{-2}$	$1.5e^{-3}$	$6.5e^{-5}$	N/A	N/A
0th Order Hold + Correction	$2.5e^{-2}$	$2.5e^{-4}$	$2.5e^{-6}$	$6.3e^{-4}$	$2.0e^{-5}$	N/A
0th Order Hold gain	288.2	61.4	604.2	9.7	N/A	N/A
1st Order Hold	$5.7e^{-2}$	$5.5e^{-4}$	$5.5e^{-6}$	$4.2e^{-4}$	$1.3e^{-5}$	N/A
1st Order Hold + Correction	$4.2e^{-3}$	$4.0e^{-6}$	$4.0e^{-9}$	$1.3e^{-3}$	$1.3e^{-4}$	$1.3e^{-5}$
1st Order Hold gain	13.7	137.7	1385.4	3.0	10.1	N/A
2nd Order Hold	$4.2e^{-2}$	$3.2e^{-5}$	$3.2e^{-8}$	$6.5e^{-4}$	$6.6e^{-5}$	$6.6e^{-6}$
2nd Order Hold + Correction	$2.4e^{-3}$	$2.1e^{-7}$	$2.0e^{-11}$	$1.4e^{-3}$	$2.6e^{-4}$	$4.5e^{-5}$
2nd Order Hold gain	17.5	156.3	1584.3	2.1	3.9	6.7
3rd Order Hold	N/A	$2.8e^{-6}$	$2.7e^{-10}$	$6.1e^{-4}$	$1.4e^{-4}$	$2.5e^{-5}$
3rd Order Hold + Correction	$1.8e^{-3}$	$1.4e^{-8}$	$1.4e^{-13}$	$1.4e^{-3}$	$3.9e^{-4}$	$9.6e^{-5}$
3rd Order Hold gain	N/A	192.1	1922.3	2.3	2.9	3.9
4th Order Hold	N/A	$3.0e^{-7}$	$2.8e^{-12}$	$1.5e^{-4}$	$1.5e^{-4}$	$4.8e^{-5}$
4th Order Hold + Correction	$1.8e^{-3}$	$6.1e^{-10}$	$6.4e^{-16}$	$1.3e^{-3}$	$4.6e^{-4}$	$1.6e^{-4}$
4th Order Hold gain	N/A	485.0	4317.9	8.3	3.0	3.3

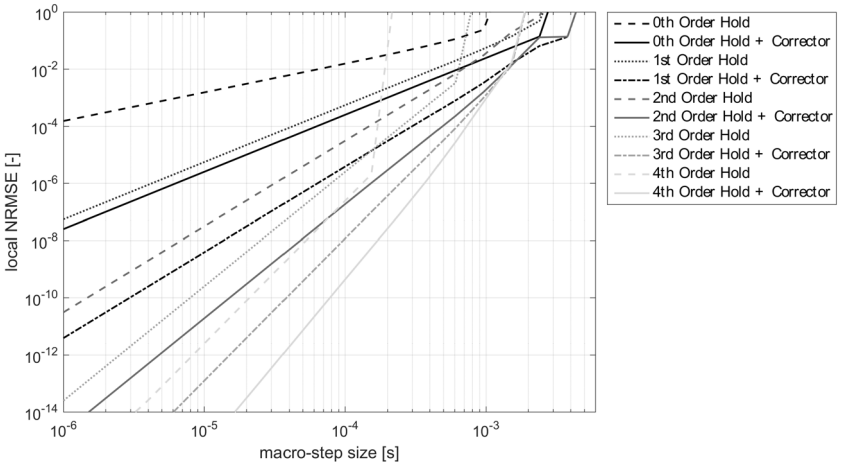


Fig. 8 Error convergence for different polynomial extrapolation orders with and without model-based correction

given polynomial extrapolation order can greatly reduce the error of the cosimulation for a given macrostep size or allow substantially larger macrostep sizes for a desired cosimulation error. For example, to achieve a local error of 10^{-3} for ZOH without model-based correction a macrostep size of $\Delta T \approx 7 \cdot 10^{-6}$ s is necessary, which can be reduced to $\Delta T \approx 2 \cdot 10^{-4}$ s by adding model-based correction.

Finally, model-based correction also stabilizes the cosimulation for large macrostep sizes, especially for high polynomial extrapolation orders (e.g., 3rd-order hold or 4th-order hold), which otherwise suffer instability due to Runge's phenomenon. It can be seen that, for example, for 4th-order hold the local error increases rapidly for macrostep sizes of $\Delta T >\sim 1.8 \cdot 10^{-4}$ s.

5 Application to a nonlinear two-mass oscillator

While the use of linear test models for the analysis of a cosimulation method is common practice since they represent a well-understood and well-defined subset of possible test models, the validity with respect to nonlinear systems is highly dependent on how well these systems can be linearized. Since the model-based corrector approach is derived by assuming a linearized coupled system for the duration of the current macrostep (see Sect. 2) the applicability of the algorithm to nonlinear systems depends on how well this linearization captures the behavior of the actual system in the current macrostep. This raises the question of the robustness of the model-based corrector regarding imperfect model knowledge, e.g., due to nonlinearity in the system, and is beyond the scope of this article. Therefore, the model-based corrector approach is applied to a nonlinear test model as an example in this section.

To this end, the two-mass oscillator introduced in Sect. 3.1 is used as a basis, but the constant coupling damping d_k is replaced by the following calculation rule resulting in a nonlinear spring damper characteristic:

$$c_k\left(x_2^{II}, u_2^{II}\right) = \max\left(\min\left(d_k \cdot |x_2^{II} - u_2^{II}|^{n_d-1}, d_{k,max}\right), d_{k,min}\right), \tag{43}$$

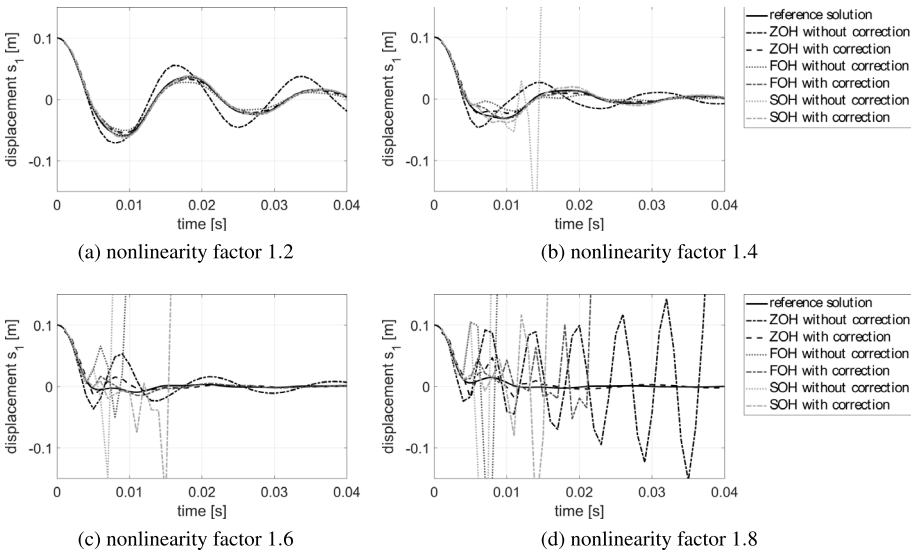
where n_d represents a "nonlinearity factor" ($n_d = 1$ results in a linear system). The values for c_k and d_k are fixed to moderate values of $c_k = 2 \cdot 10^5$ and $d_k = 5 \cdot 10^2$, respectively, and the macrostep size is set to $\Delta T = 1e-3$, while the nonlinearity factor n_d is varied in the range $n_d \in [1\ 2]$. The limits of the coupling damping are set to $d_{k,min} = 1 \cdot 10^1$ and $d_{k,max} = 1 \cdot 10^4$.

Figure 9 shows the displacement s_1 of mass $m1$ for polynomial extrapolation from 0th order to 2nd order with and without model-based correction for several values of the nonlinearity factor n_d. For these cases the global error $e_{g,NRMSE}$ is also listed in Table 4. It can be seen that for ZOH the improvement due to model-based correction becomes smaller with increasing nonlinearity factor n_d, up to the point where ZOH without correction becomes unstable and adding model-based correction stabilizes the cosimulation. Therefore, model-based correction significantly improves the stability for high nonlinearity factors in this case. The same effect can be observed for higher-order polynomial extrapolation, where model-based correction can stabilize the cosimulation for higher nonlinearity factors up to a certain point. Also, note that the global error grows with increasing nonlinearity factor n_d for all cosimulation methods presented. In addition, it can be seen that increasing nonlinearity proves more problematic for higher polynomial orders in this example, since polynomial extrapolation also assumes some degree of linearity in the system.

As a result, model-based correction seems perfectly applicable also for nonlinear problems, but of course a detailed analysis in this area still needs to be carried out.

6 Conclusion and outlook

This article demonstrates that the model-based corrector approach presented in [17] for ZOH can be easily applied to other extrapolation methods, for example to higher-order polyno-



(a) nonlinearity factor 1.2 (b) nonlinearity factor 1.4

(c) nonlinearity factor 1.6 (d) nonlinearity factor 1.8

Fig. 9 Displacement s_1 of mass m_1 for different nonlinearity factors

Table 4 Global error of the nonlinear two-mass oscillator for different nonlinearity factors and 0th-order to 2nd-order polynomial extrapolation with and without model-based correction

nonlinearity factor n_d	1.0	1.2	1.4	1.6	1.8	2.0
0th Order Hold	$1.1e^{0}$	$9.5e^{-1}$	$1.2e^{0}$	$5.2e^{0}$	$4.2e^{2}$	$2.6e^{2}$
0th Order Hold + Correction	$4.6e^{-2}$	$8.9e^{-2}$	$2.4e^{-1}$	$1.0e^{0}$	$3.3e^{0}$	$3.6e^{0}$
0th Order Hold accuracy gain	22.7	10.7	5.3	5.2	128.8	72.2
1st Order Hold	$1.7e^{-1}$	$2.1e^{-1}$	$5.2e^{-1}$	N/A	N/A	N/A
1st Order Hold + Correction	$6.9e^{-2}$	$9.7e^{-2}$	$1.9e^{-1}$	$7.4e^{-1}$	N/A	N/A
1st Order Hold accuracy gain	2.4	2.1	2.7	N/A	N/A	N/A
2nd Order Hold	$7.5e^{-2}$	$1.4e^{-1}$	N/A	N/A	N/A	N/A
2nd Order Hold + Correction	$5.6e^{-2}$	$7.5e^{-2}$	$3.3e^{-1}$	N/A	N/A	N/A
2nd Order Hold accuracy gain	1.3	1.85	N/A	N/A	N/A	N/A

mial extrapolation like FOH or SOH. By modeling the cosimulation process via a linear recurrence equation of the form $\mathbf{z}_{n+1} = \boldsymbol{\Phi}\mathbf{z}_n$ where the coupled system with the respective cosimulation method is represented by a propagation matrix $\boldsymbol{\Phi}$, the stability, accuracy and error convergence of this cosimulation method are efficiently analyzed without any side effects from numerical solvers. Through the application of this methodology to the well-known two-mass oscillator test model, which is coupled with ZOH, FOH, and SOH extrapolation with and without model-based correction, respectively, it can be seen that model-based correction can significantly improve the stability, accuracy, and error convergence of the underlying extrapolation method (see key takeaways below).

While the model-based correction algorithm itself consists only of a series of matrix operations and is therefore highly efficient (provided the number of outputs is not too large),

obtaining the necessary subsystem Jacobians can be computationally expensive since either the subsystem has to calculate its Jacobians itself or they have to be calculated through a system-identification approach like MOESP. Depending on how fast the subsystem dynamics change, it might be possible to keep the Jacobians for several macrosteps, which can significantly reduce the computation cost. One possible approach is to check how well a linear model using the "old" Jacobians predicts the current macrostep, for example by using a so-called extended state-space model (ESSM [27]), and refresh or keep the Jacobians based on this prediction error (see [17] for details).

Nevertheless, in some cases, there can be a significant overhead compared to simpler coupling algorithms such as plain polynomial extrapolation. Therefore, model-based correction is best used when either the macrosteps of the subsystems require a high computational effort themselves, in which case model-based correction can speed up the simulation if it allows larger macrostep sizes, or the subsystems have technical limitations regarding minimum macrostep size or maximum input extrapolation order, in which case model-based correction can still allow accurate results although the trivial solution (smaller macrostep sizes, higher extrapolation order) is not feasible. To conclude, the overhead due to the model-based corrector approach has to be justified, but if it is, the approach can significantly improve the quality of the cosimulation.

The current work focuses on an indepth robustness analysis of the method with respect to imperfect model knowledge due to system identification and/or underlying nonlinear subsystems. A case study using different examples from the literature and industry will also be conducted in the near future. In addition, the state error correction is based on a correction offset that is calculated only from the values of the past and current corrected outputs. Future research will therefore try to incorporate model knowledge, which is required for the output correction in any case, into the correction of the states as well.

Key takeaways:

- Model-based correction can be easily extended to other extrapolation methods.
- Modeling the cosimulation process as a linear recurrence equation allows the efficient analysis of cosimulation methods.
- Model-based correction offers significant stability and accuracy improvement in some cases (mainly high stiffness, low damping combinations, or large macrostep sizes).
- In other cases, model-based correction still offers some stability and accuracy improvement (mainly low stiffness, high damping combinations, or balanced stiffness and damping).
- Lower-order polynomial extrapolation generally benefits more from model-based correction.
- The improvement in accuracy due to model-based correction is generally more pronounced than the stability improvement.
- Error convergence is increased by one order when model-based correction is used.
- Model-based correction offers a significant improvement of the necessary macrostep size or produced error when high accuracy requirements or small macrostep sizes are given.
- Future research focuses on robustness with respect to imperfect model knowledge due to system identification and/or underlying nonlinear subsystems and on improving the state error correction.

Funding Note Open access funding provided by Graz University of Technology.

Declarations

Conflict of interest The authors declare that they have no conflict of interest.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by/4.0/.

References

1. Andersson, C.: Methods and tools for co-simulation of dynamic systems with the functional mock-up interface. Ph.D. thesis (2016)
2. Arnold, M.: Multi-Rate Time Integration for Large Scale Multibody System Models pp. 1–10. Springer, Dordrecht (2007)
3. Arnold, M.: Numerical stabilization of co-simulation techniques the ode case. In: Working Document MODELISAR, sWP 200 - 203 (2011)
4. Arnold, M., Günther, M.: Preconditioned dynamic iteration for coupled differential-algebraic systems. BIT Numer. Math. **41**, 1–25 (2001)
5. Arnold, J., Einarsson, G., Schütte, A.: Multibody simulation of an aeroelastic delta wing in roll maneuver. In: Proceedings of ICAS 2006 (2006)
6. Arnold, M., Clauß, C., Schierz, T.: Error Analysis and Error Estimates for Co-Simulation in FMI for Model Exchange and Co-Simulation v2.0 pp. 107–125. Springer, Berlin (2014)
7. Benedikt, M., Watzenig, D., Hofer, A.: Modelling and analysis of the non-iterative coupling process for co-simulation. Math. Comput. Model. Dyn. Syst. **19**(5), 451–470 (2013)
8. Blochwitz, T., Otter, M., Åkesson, J., Arnold, M., Clauss, C., Elmqvist, H., Friedrich, M., Junghanns, A., Mauss, J., Neumerkel, D., Olsson, H., Viel, A.: Functional mockup interface 2.0: the standard for tool independent exchange of simulation models. In: Proceedings of the 9th International MODELICA Conference (2012). https://doi.org/10.3384/ecp12076173
9. Busch, M., Schweizer, B.: Numerical stability and accuracy of different co-simulation techniques: analytical investigations based on a 2-dof test model. In: Proceedings of the 1st Joint International Conference on Multibody System Dynamics (2010)
10. Busch, M., Schweizer, B.: Stability of co-simulation methods using Hermite and Lagrange approximation techniques. In: Proceedings of ECCOMAS Thematic Conference on Multibody Dynamics, Brussels, pp. 1–10 (2011)
11. Friedrich, M.: Parallel co-simulation for mechatronic systems. Ph.D. thesis (2011)
12. Friedrich, M., Schneider, M., Ulbrich, H.: A parallel co-simulation for mechatronic systems. In: IMSD, First Joint International Conference on Multibody System Dynamics, pp. 1–10 (2010)
13. Genser, S., Benedikt, M.: A Pre-Step Stabilization Method for Non-iterative Co-Simulation and Effects of Interface-Jacobians Identification. Advances in Intelligent Systems and Computing (2018)
14. González, F., González, M., Cuadrado, J.: Weak coupling of multibody dynamics and block diagram simulation tools. In: ASME 2009 International Design Engineering Technical Conferences and Computers and Information in Engineering Conference (2009)
15. González, F., Naya, M., Luaces, A., González, M.: On the effect of multirate co-simulation techniques in the efficiency and accuracy of multibody system dynamics. Multibody Syst. Dyn. **25**(4), 461–483 (2011)
16. Günther, M., Kværnø, A., Rentrop, P.: Multirate partitioned Runge-Kutta methods. BIT Numer. Math. **41**(3), 504–514 (2001)
17. Haid, T., Stettinger, G., Watzenig, D., Benedikt, M.: A model-based corrector approach for explicit co-simulation using subspace identification. In: The 5th Joint International Conference on Multibody System Dynamics (2018)
18. Hairer, E., Nørsett, S.P., Wanner, G.: Solving Ordinary Differential Equations I, vol. 8, 2nd edn. Springer, Berlin (1993)
19. Katayama, T.: Subspace Methods for System Identification, 1st edn. (2005). ISBN: 978-1-84628-158-7
20. Kübler, R., Schiehlen, W.: Modular simulation in multibody system dynamics. Multibody Syst. Dyn. **4**(2–3), 107–127 (2000)

21. Kübler, R., Schiehlen, W.: Two methods of simulator coupling. Math. Comput. Model. Dyn. Syst. **6**(2), 93–113 (2000)
22. Lelarasmee, E., Ruehli, A.E., Sangiovanni-Vincentelli, A.L.: The waveform relaxation method for time-domain analysis of large scale integrated circuits. IEEE Trans. Comput.-Aided Des. Integr. Circuits Syst. **1**, 131–145 (1982)
23. Liebig, S., Helduser, S., Stüwing, M., Dronka, S.: Die modellierung und simulation gekoppelter mechanischer und hydraulischer systeme (2001)
24. Ljung, L.: System Identification Theory for the User, 2nd ed. Prentice Hall, New York (1999)
25. Overschee, P.V., Moor, B.D.: N4sid: subspace algorithms for the identification of combined deterministic-stochastic systems. Automatica **30**(1), 75–93 (1994). https://doi.org/10.1016/0005-1098(94)90230-5
26. Rowell, D.: Time-Domain Solution of LTI State Equations (2002)
27. Ruscio, D.D.D.: Model based predictive control: an extended state space approach. In: Proceedings of the 36th IEEE Conference on Decision and Control, vol. 4, pp. 3210–3217 (1997). https://doi.org/10.1109/CDC.1997.652337
28. Sadjina, S.: Energy conservation and coupling error reduction in non-iterative co-simulations (2016). arXiv:1606.05168
29. Sadjina, S., Kyllingstad, L., Skjong, S., Pedersen, E.: Energy conservation and power bonds in co-simulations: non-iterative adaptive step size control and error estimation (2016). arXiv:1602.06434
30. Schierz, T., Arnold, M., Clauss, C.: Co-simulation with communication step size control in an fmi compatible master algorithm. In: Proceedings of the 9th International MODELICA Conference (2012)
31. Schweizer, B.: Explicit and implicit co-simulation methods: stability and convergence analysis for different solver coupling approaches. J. Comput. Nonlinear Dyn. **10**(5), 051007 (2015)
32. Schweizer, B., Lu, D.: Semi-implicit co-simulation approach for solver coupling. Arch. Appl. Mech. **84**, 1739–1769 (2014)
33. Stettinger, G., Zehetner, J., Benedikt, M., Thek, N.: Extending Co-Simulation to the Real-Time Domain. SAE International, SAE 2013 World Congress & Exhibition, Detroit, Mich. (2013). https://doi.org/10.4271/2013-01-0421
34. Stettinger, G., Horn, M., Benedikt, M., Zehetner, J.: A model-based approach for prediction-based interconnection of dynamic systems. In: 53rd IEEE Conference on Decision and Control (2014)

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.